Network Programming
Distributed ML and LLM Systems

Minchen Yu

SDS@CUHK-SZ

Spring 2026

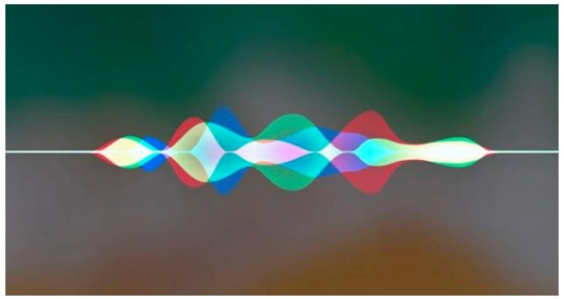
香港中文大學（深圳）
The Chinese University of Hong Kong, Shenzhen

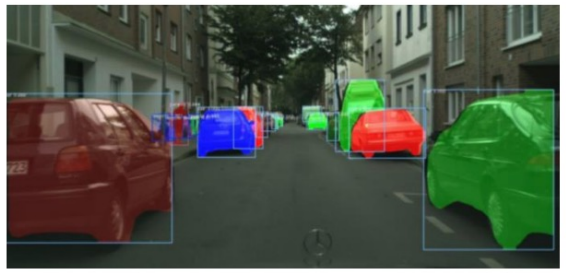
SCHOOL OF
DATA SCIENCE
數據科學學院

Outline

- Background of distributed ML
- ML training
- ML serving
- State-of-the-art LLM systems (advanced topics)

Successes of machine learning

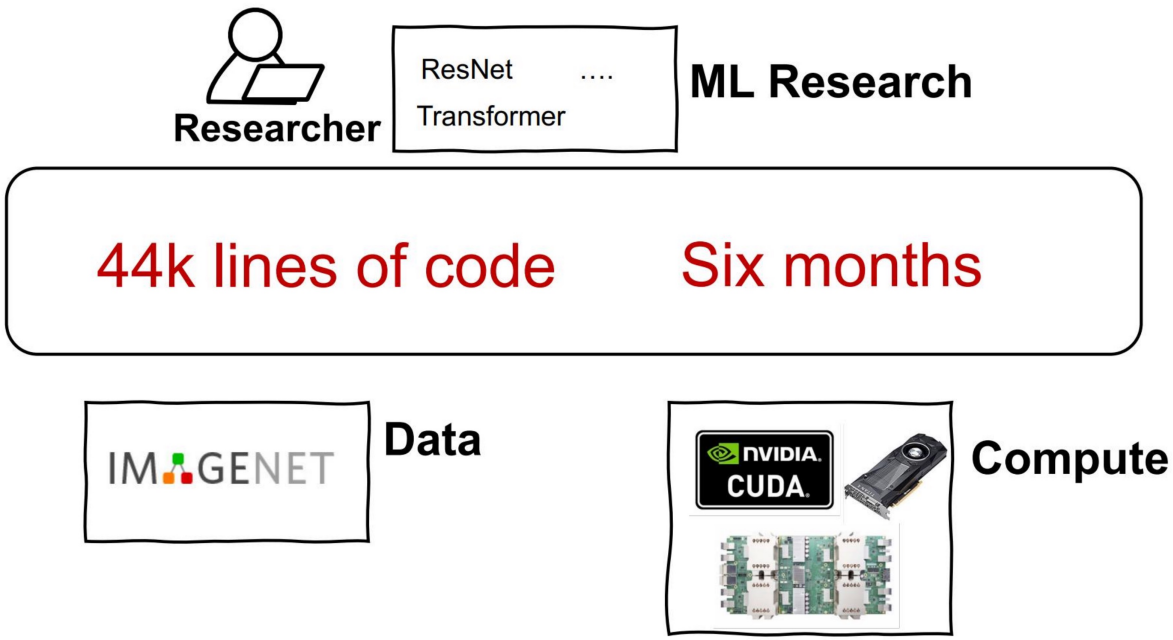




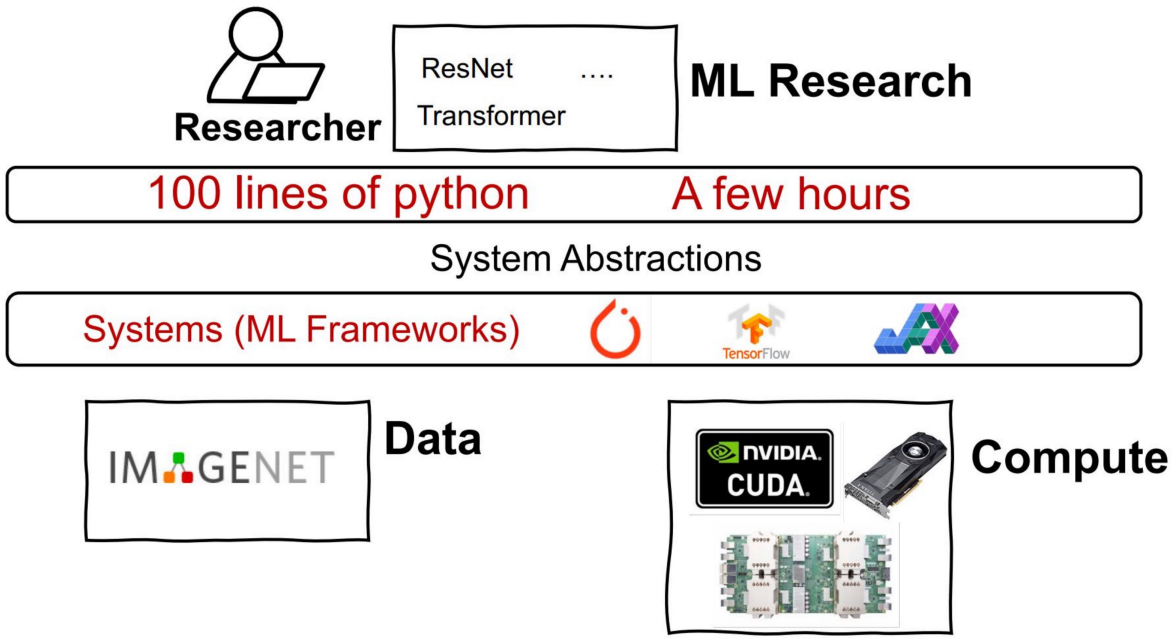
ImageGen models

A photo of an astronaut riding a horse on mars

Why ML systems?



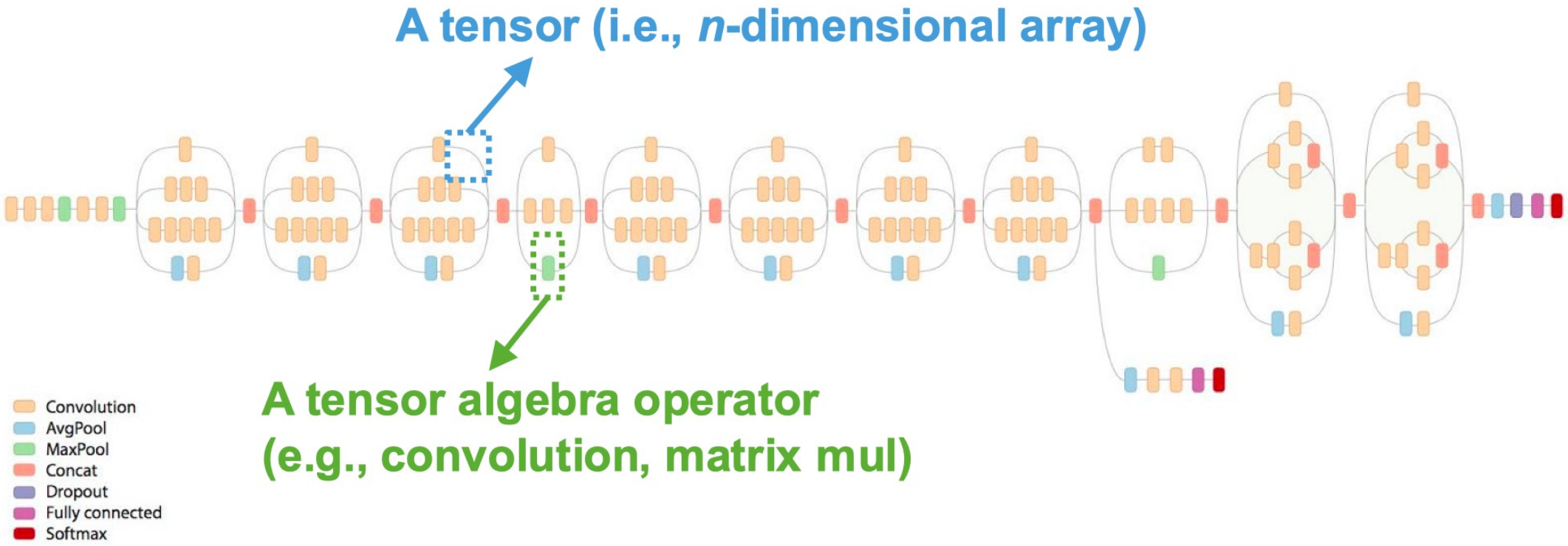
w/o ML systems



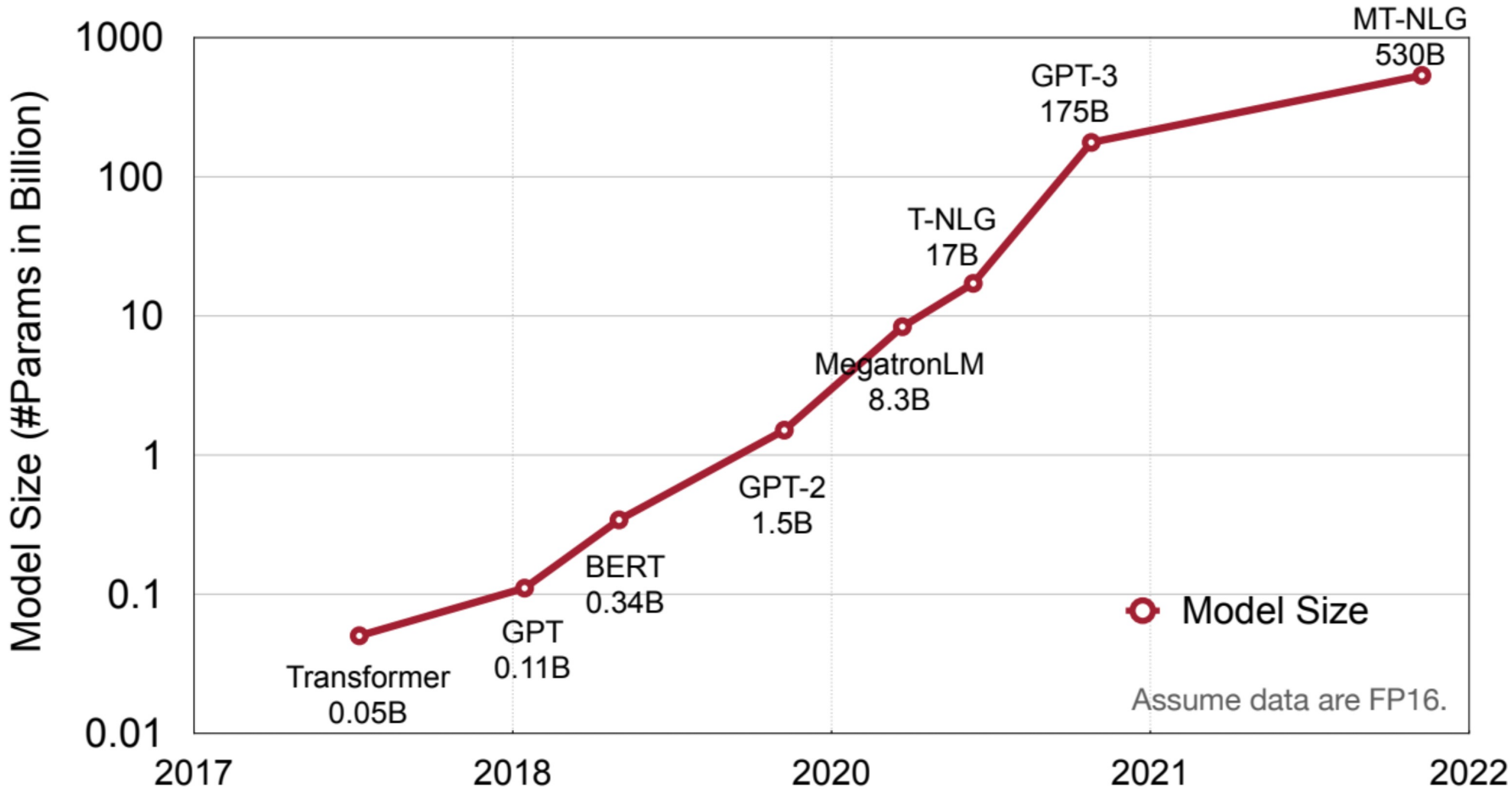
w/ ML systems

Deep neural network models

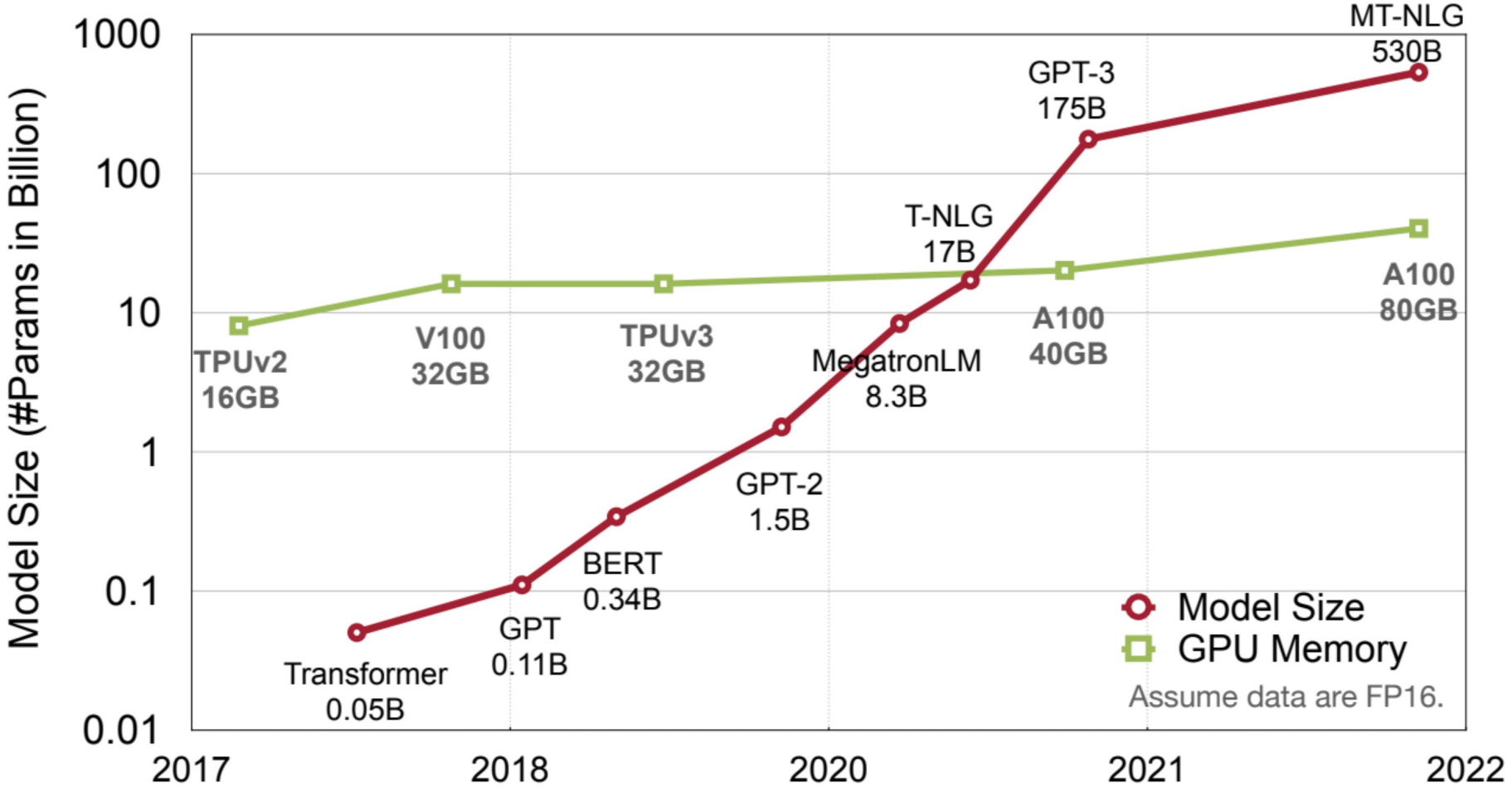
- Collection of simple trainable mathematical units that work together to solve complicated tasks



Models are getting larger



Models are getting larger



Models are getting larger

- Larger models take longer time to train

Models	#Params (M)	Training Time (GPU Hours)
ResNet-50	26	31
ResNet-101	45	44
BERT-Base	108	84
Turing-NLG 17B	17,000	TBA
GPT-3 175B	175,000	3,100,000

Measured on Nvidia A100

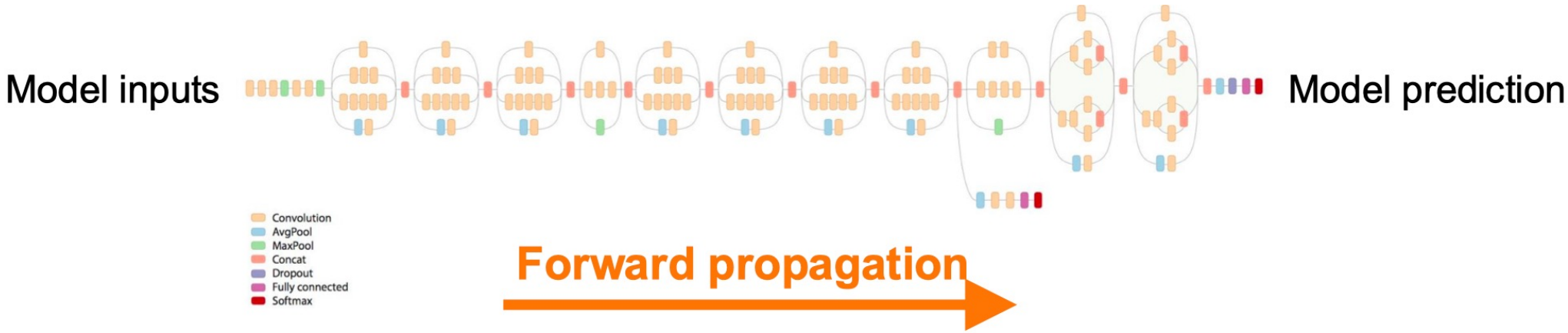
⟶ **Over 330 years!**

ML training

ML training

Train ML models through many iterations of 3 stages

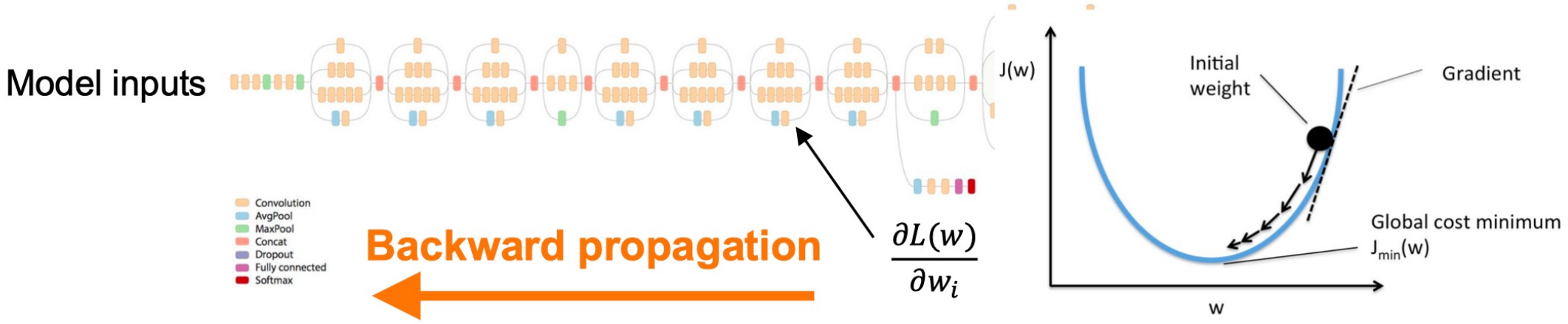
1. **Forward propagation**: apply model to a batch of input samples and run calculation through operators to produce a prediction
2. **Backward propagation**: run the model in reverse to produce a gradient for each trainable weight
3. **Weight update**: use the loss value to update model weights



ML training

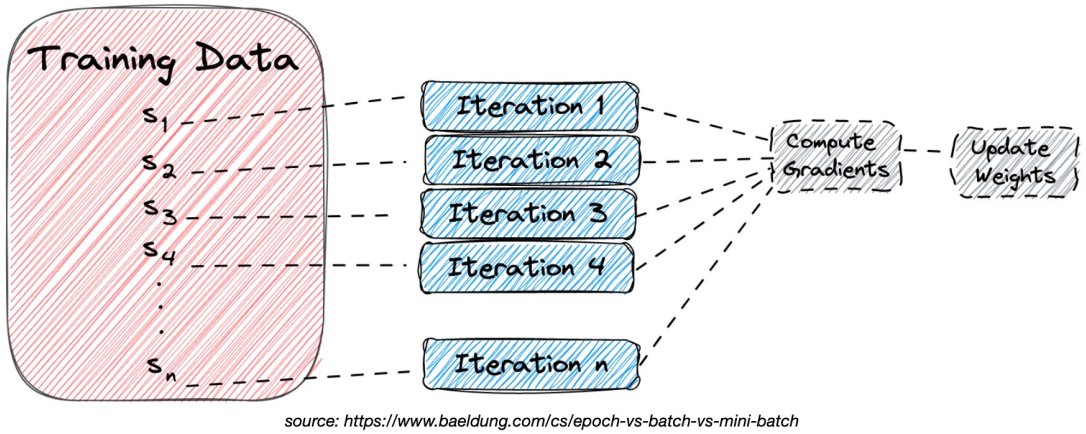
Train ML models through many iterations of 3 stages

1. **Forward propagation**: apply model to a batch of input samples and run calculation through operators to produce a prediction
2. **Backward propagation**: run the model in reverse to produce a gradient for each trainable weight
3. **Weight update**: use the loss value to update model weights



Distributed machine learning

Gradient descent for model training



source: https://www.baeldung.com/cs/epoch-vs-batch-vs-mini-batch

Gradient / backward computation

$$\theta^{(t)} = \theta^{(t-1)} + \varepsilon \cdot \nabla_{\mathcal{L}}(\theta^{(t-1)}, D^{(t)})$$

objective data

Distribute ML across parallel workers

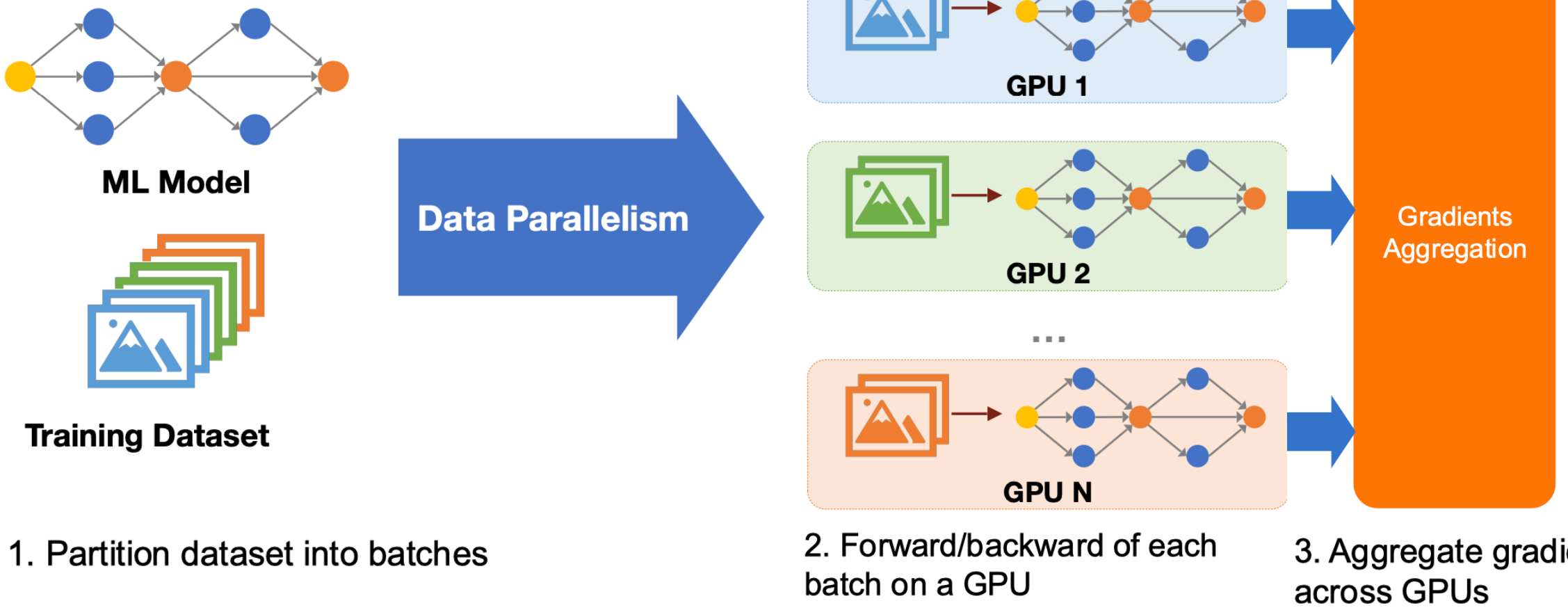
- Reduce end-to-end training time significantly

Parallelism in distributed training

- Data parallelism: split the data across workers/GPUs that keep the same model

- Model parallelism: partition the model into multiple workers with various strategies (e.g., pipeline parallelism, tensor parallelism)

Data parallelism

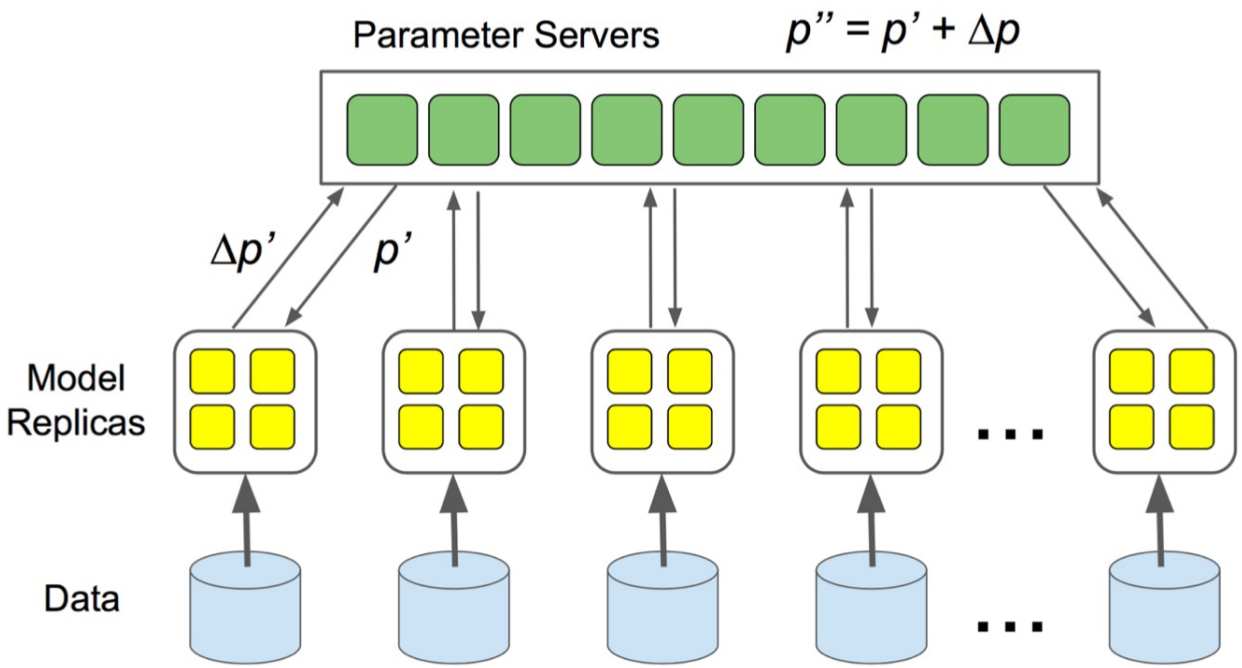


1. Partition dataset into batches

2. Forward/backward of each batch on a GPU

3. Aggregate gradients across GPUs

Parameter server



Two different roles in PS framework

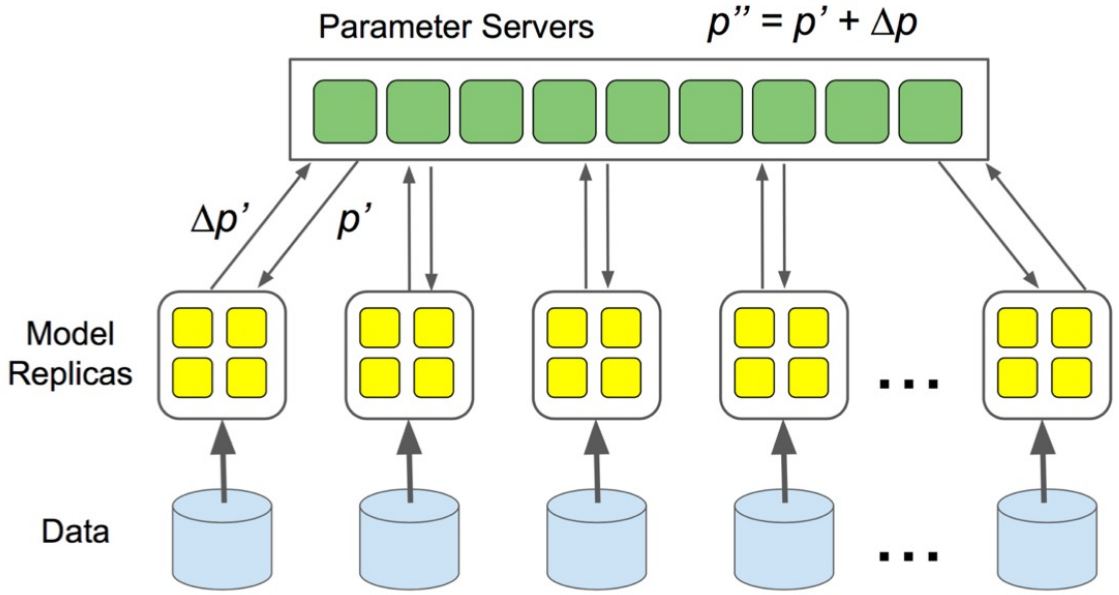
- Parameter server: receive gradients from workers and send back the aggregated results
- Workers: compute gradients using splitted dataset and send to parameter server

Any problems?

Parameter server

- **Centralized communication**: all workers communicate with parameter servers for weights update; cannot scale to large numbers of workers

- How can we decentralize communication in DNN training?

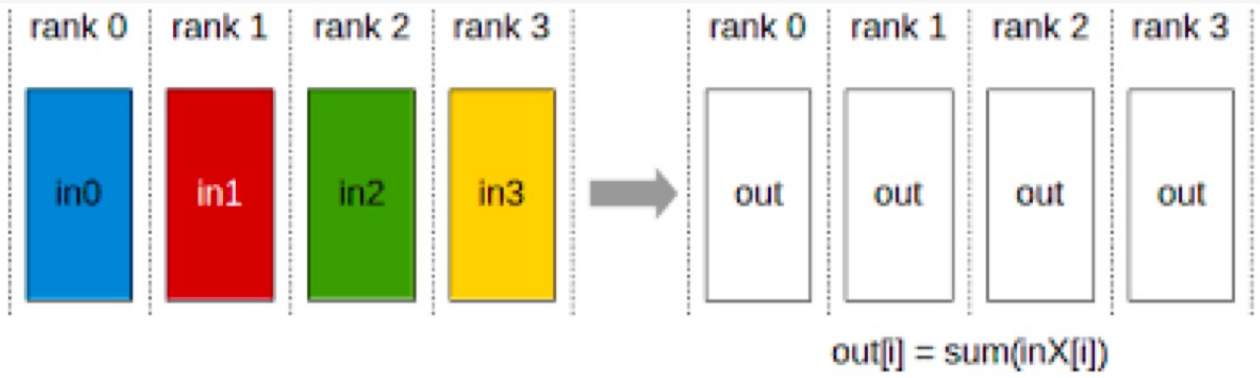


Parameter server

- **Centralized communication**: all workers communicate with parameter servers for weights update; cannot scale to large numbers of workers

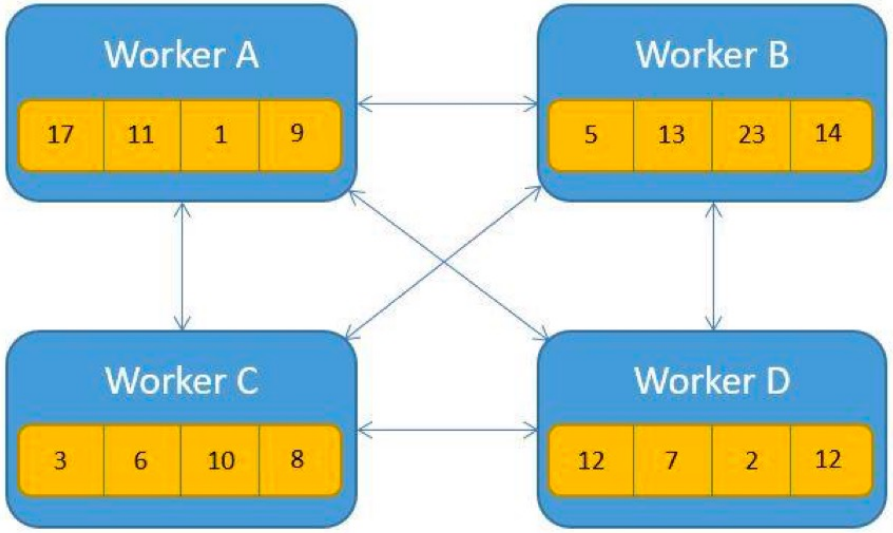
- How can we decentralize communication in DNN training?

- **AllReduce**: perform element-wise reduction across multiple devices



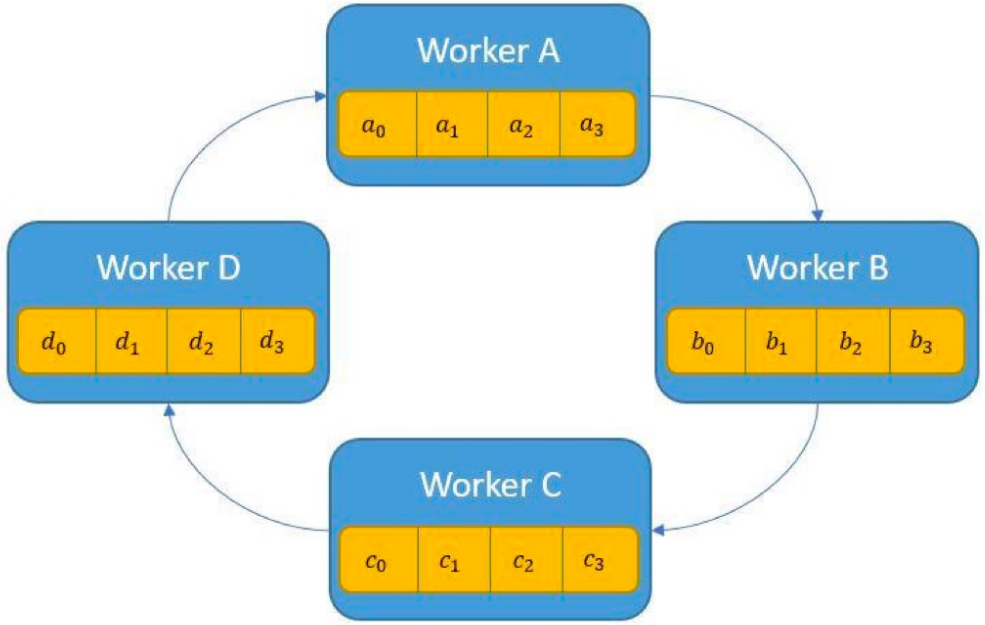
Naïve AllReduce

- Each worker can send its local gradients to all other workers
- If we have N workers and each worker contains M parameters
- Overall communication: N * (N-1) * M parameters
- **Issue**: each worker communicates with all other workers; same scalability issue as parameter server



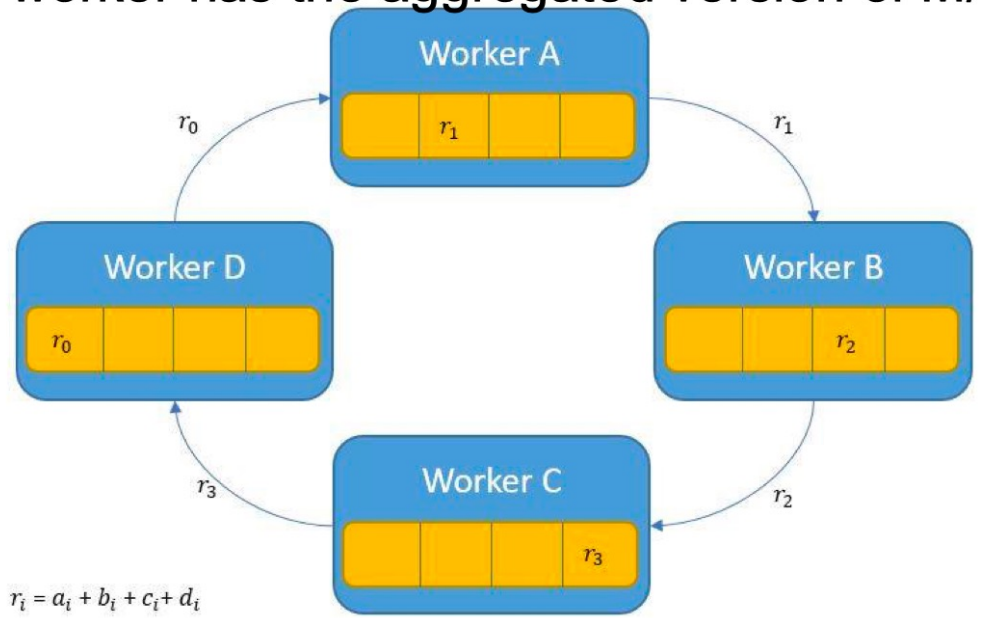
Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation): each worker send one slice (M/N parameters) to the next worker on the ring; repeat N times



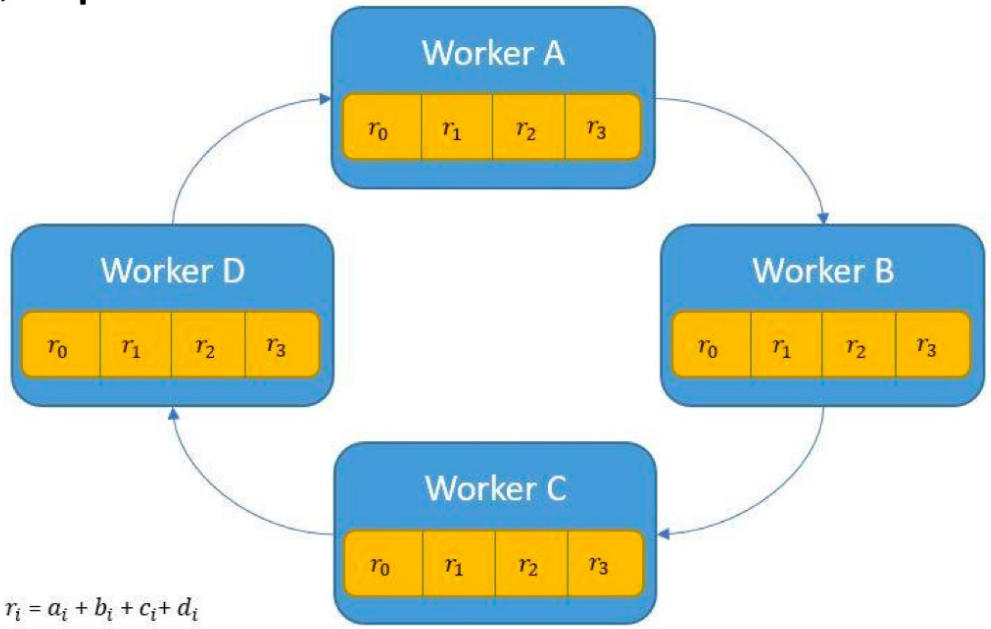
Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation): each worker send one slice (M/N parameters) to the next worker on the ring; repeat N times
- After step 1, each worker has the aggregated version of M/N parameters



Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation): each worker send one slice (M/N parameters) to the next worker on the ring; repeat N times
- Step 2 (Broadcast): each worker send one slice of aggregated parameters to the next worker; repeat N times



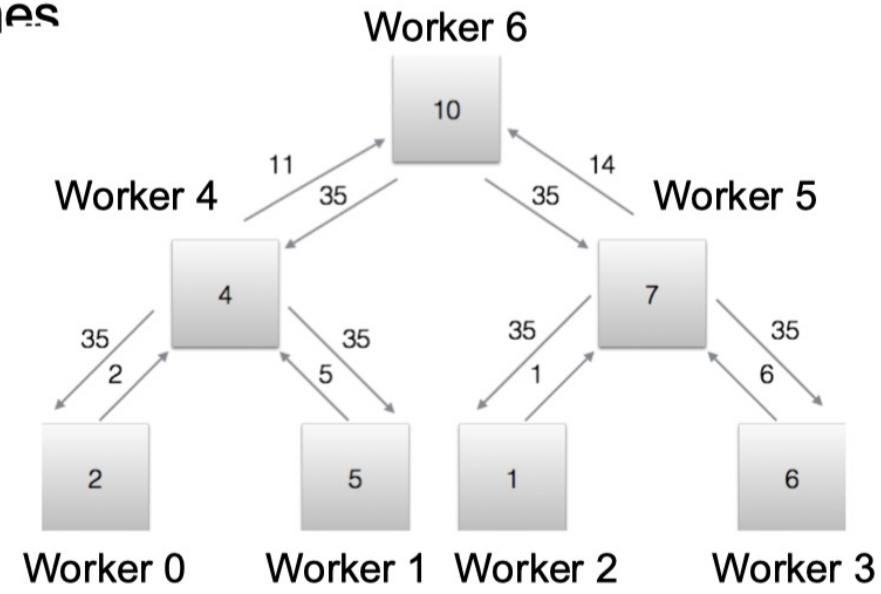
$r_i = a_i + b_i + c_i + d_i$

Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation): each worker send one slice (M/N parameters) to the next worker on the ring; repeat N times
- Step 2 (Broadcast): each worker send one slice of aggregated parameters to the next worker; repeat N times
- Overall communication: 2 * M * N parameters
  - Aggregation: M * N parameters
  - Broadcast: M * N parameters

Tree AllReduce

- Construct a tree of N workers;
- Step 1 (Aggregation): each worker sends M parameters to its parent; repeat log(N) times
- Step 2 (Broadcast): each worker sends M parameters to its children; repeat log(N) times



Tree AllReduce

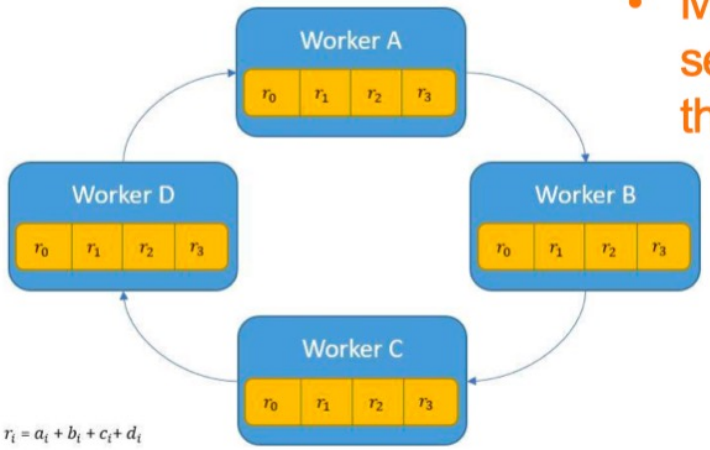
- Construct a tree of N workers;
- Step 1 (Aggregation): each worker sends M parameters to its parent; repeat log(N) times
- Step 2 (Broadcast): each worker sends M parameters to its children; repeat log(N) times
- Overall communication: 2 * N * M parameters
  - Aggregation: M * N parameters
  - Broadcast: M * N parameters

Comparison of AllReduce methods

	Parameter Server	Naïve AllReduce	Ring AllReduce	Tree AllReduce
Overall communication	$2 \times N \times M$	$N^2 \times M$	$2 \times N \times M$	$2 \times N \times M$

Question: Ring AllReduce is more efficient and scalable then Tree AllReduce and Parameter Server, why?

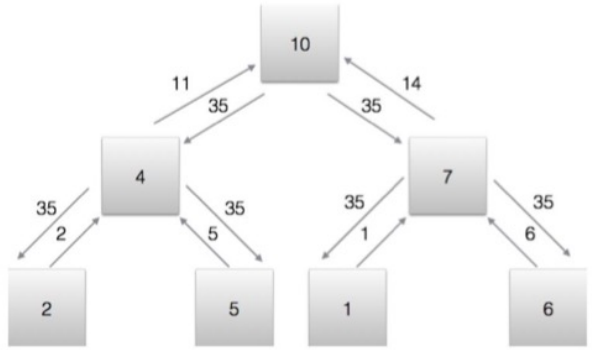
AllReduce vs. Parameter Server



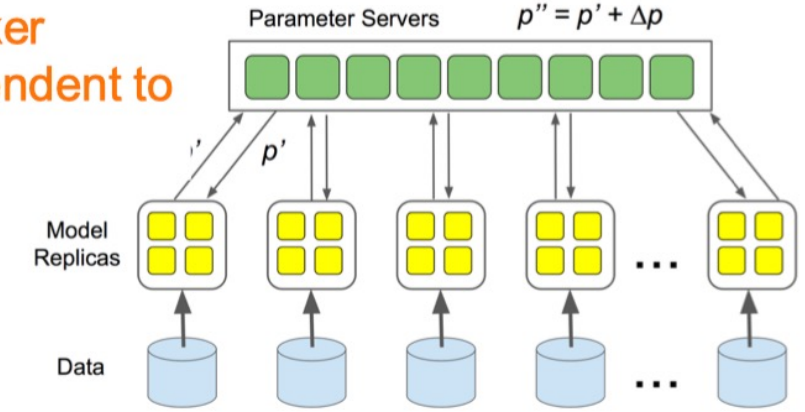
Each worker sends **M/N** parameters per iteration; repeat for **2*N** iterations
Latency: M/N * (2*N) / bandwidth

Ring AllReduce:

- Best latency
- Balanced workload across workers
- More scalable since each worker sends 2*M parameters (independent to the number of workers)



Each worker sends **M** parameters per iteration; repeat for **2*log(N)** iterations
Latency: M * 2 * log(N) / bandwidth

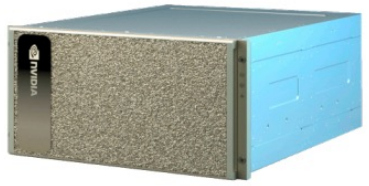


All workers send **M** parameters to parameter servers and receive **M** parameters from servers
Latency: M * N / bandwidth

Model parallelism

Data parallelism (solely) cannot train large models. A single GPU cannot fit the model into its memory.

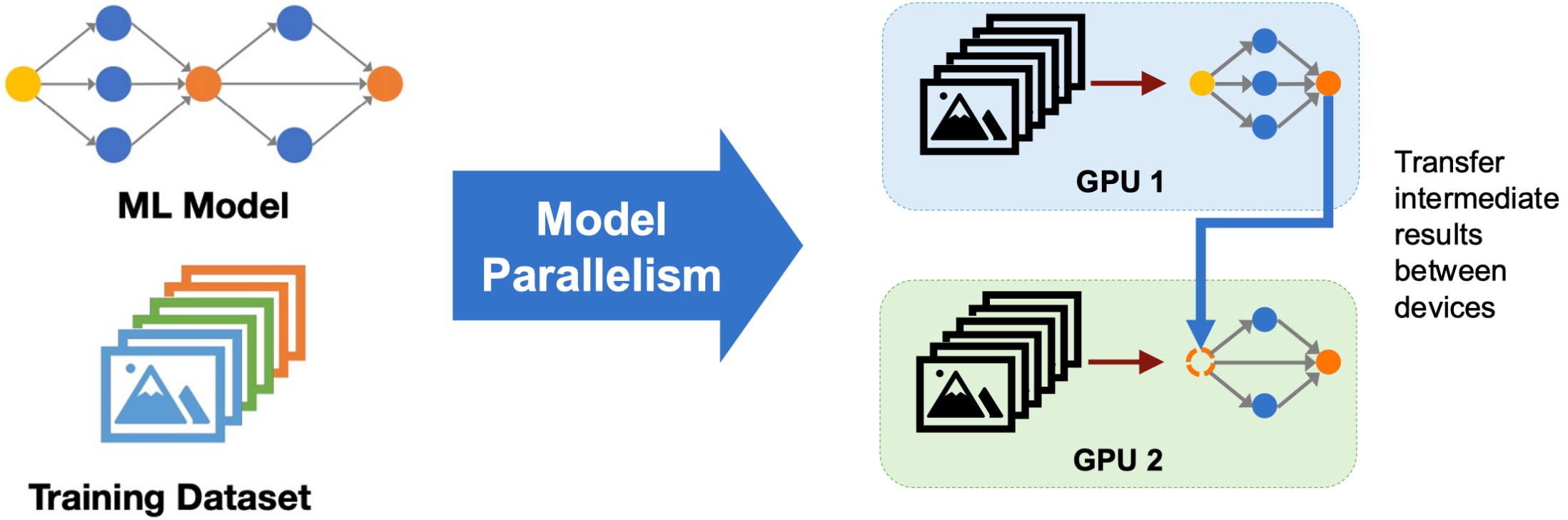
175B * 16 Bits
= 350GB Memory



Nvidia A100 80GB

Partition the model!

Model parallelism

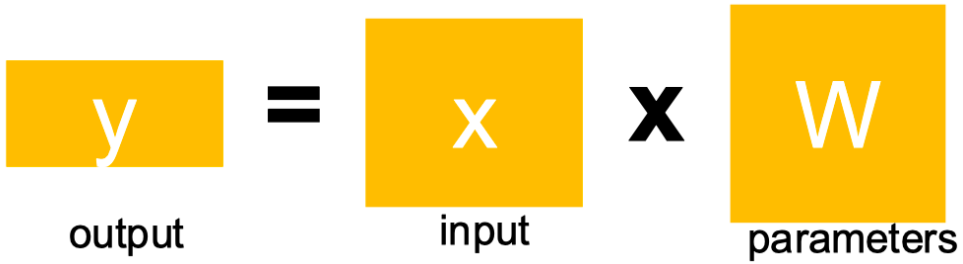


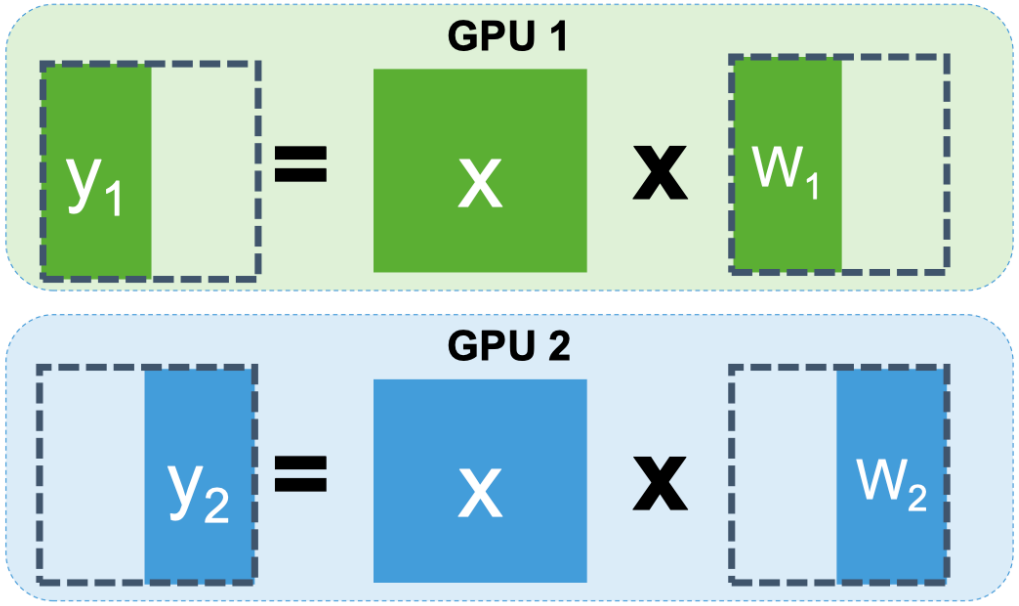
Model parallelism

- Tensor model parallelism
- Pipeline model parallelism

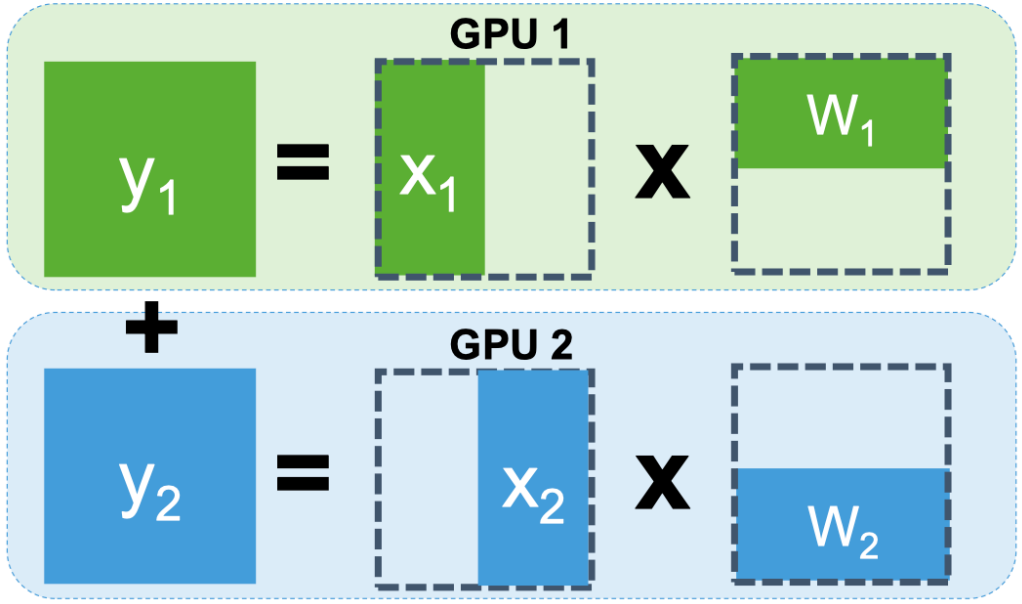
Tensor model parallelism

- Partition parameters/gradients ***within*** a layer





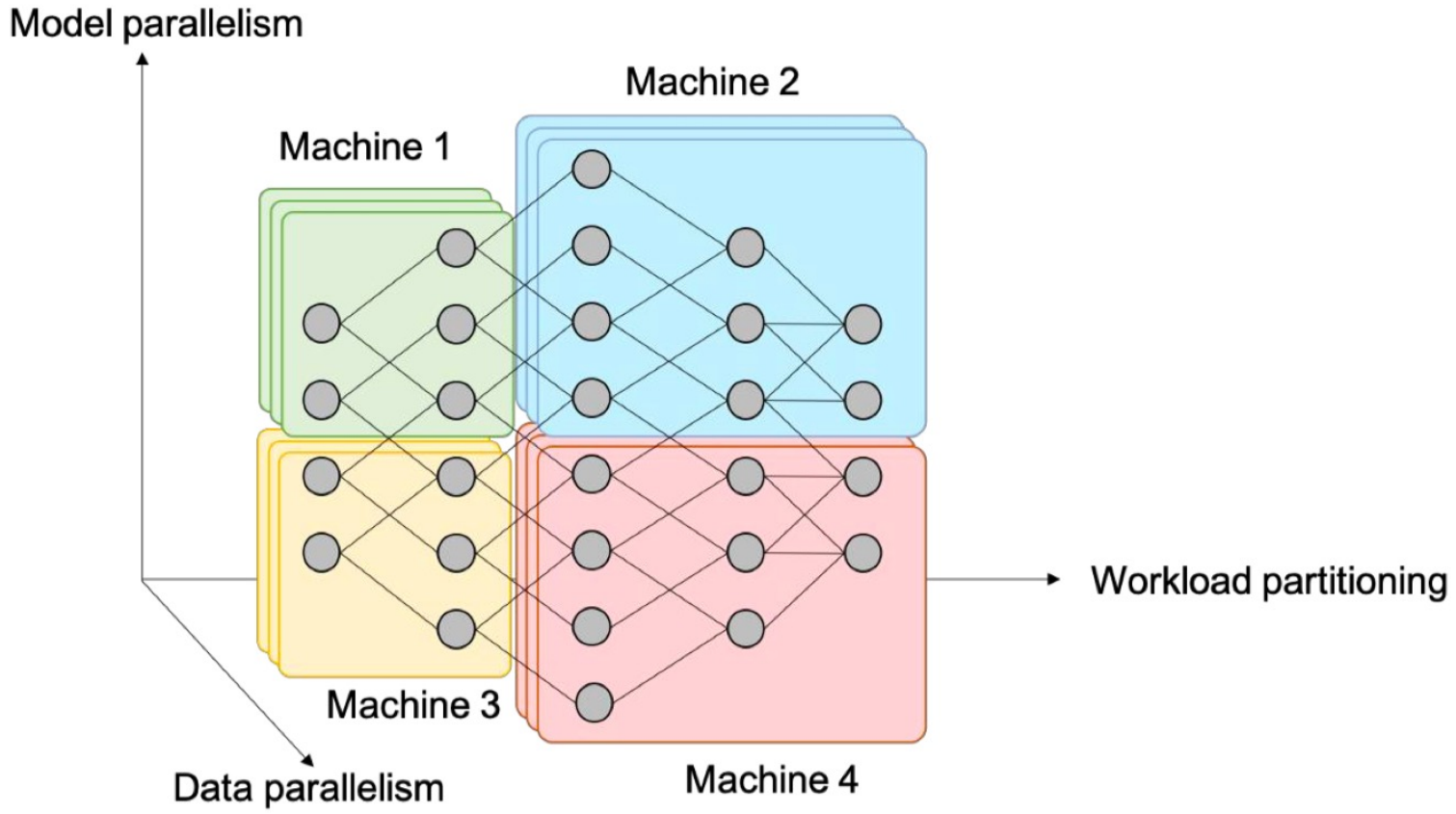
Tensor Model Parallelism (partition output)



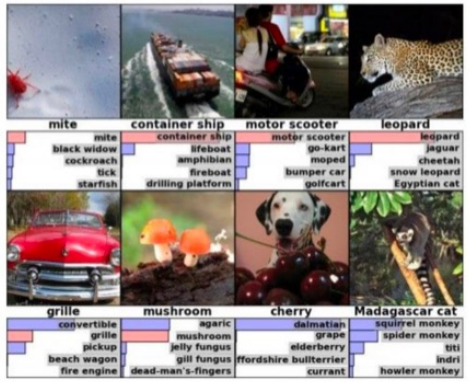
Tensor Model Parallelism (reduce output)

$y = y1 + y2$

Combine data and model parallelism

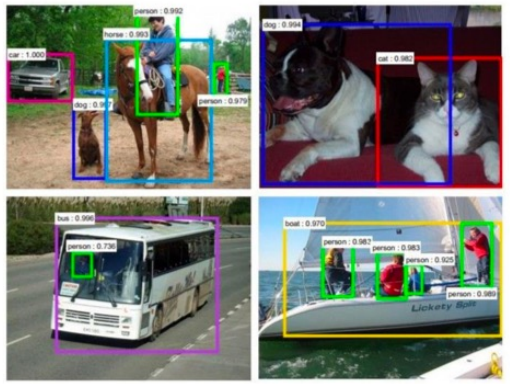


Example: Convolutional Neural Networks

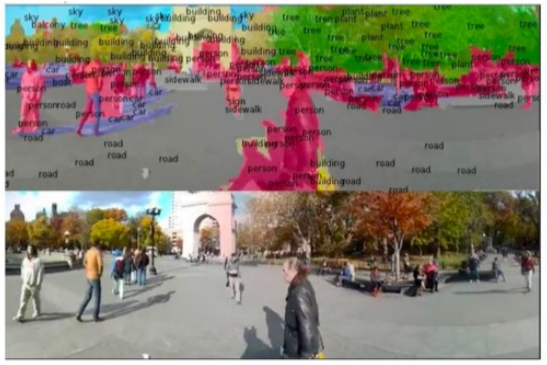


Classification

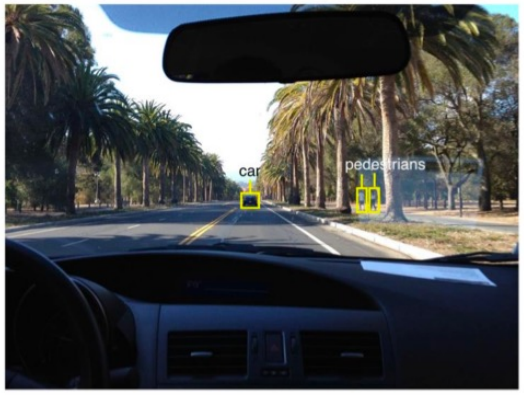
Retrieval



Detection



Segmentation

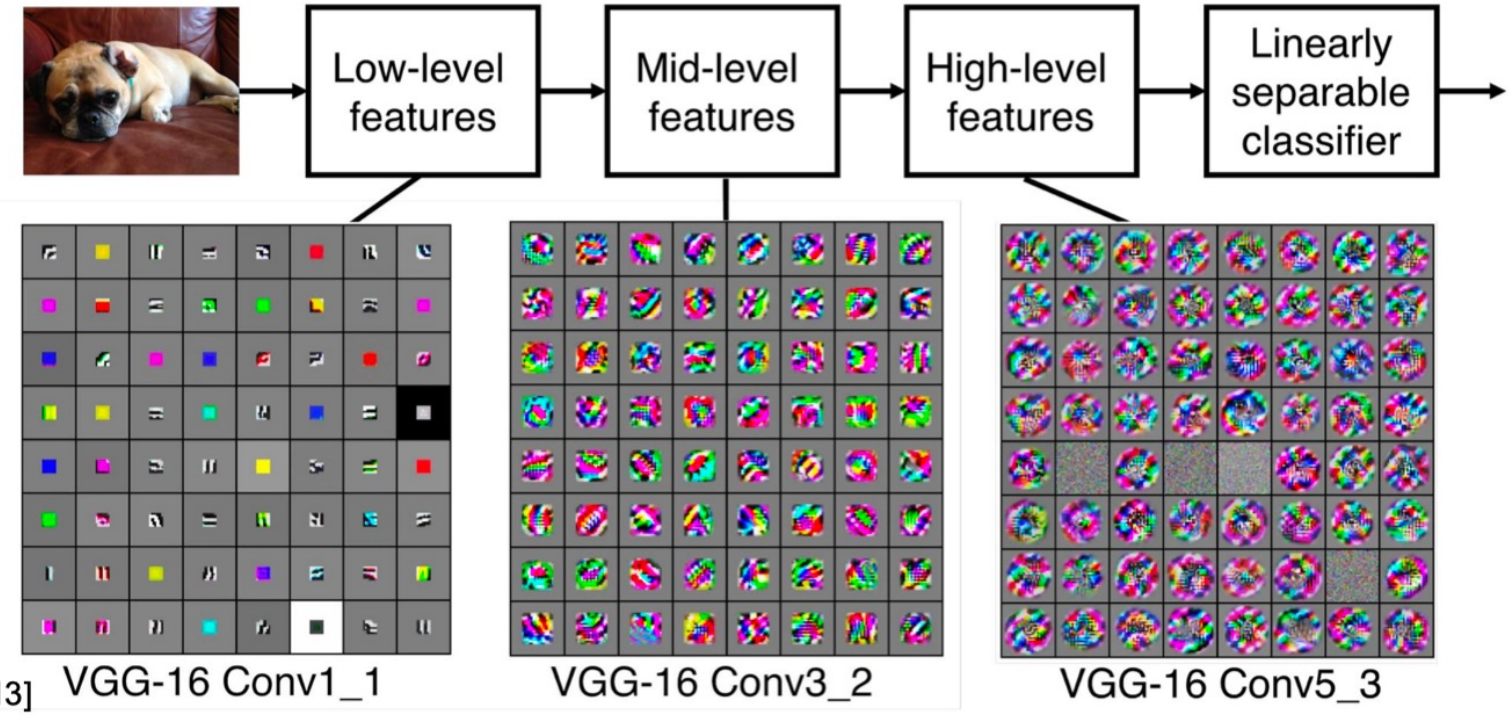


Self-Driving

Synthesis

CNNs

- A sequence of convolutional layers, interspersed by pooling, normalization, and activation functions



[Zeiler and Fergus 2013]

Parallelizing CNNs

- Convolutional layers
  - 90-95% of the computation
  - 5% of the parameters
  - Very large intermediate activations

Data parallelism

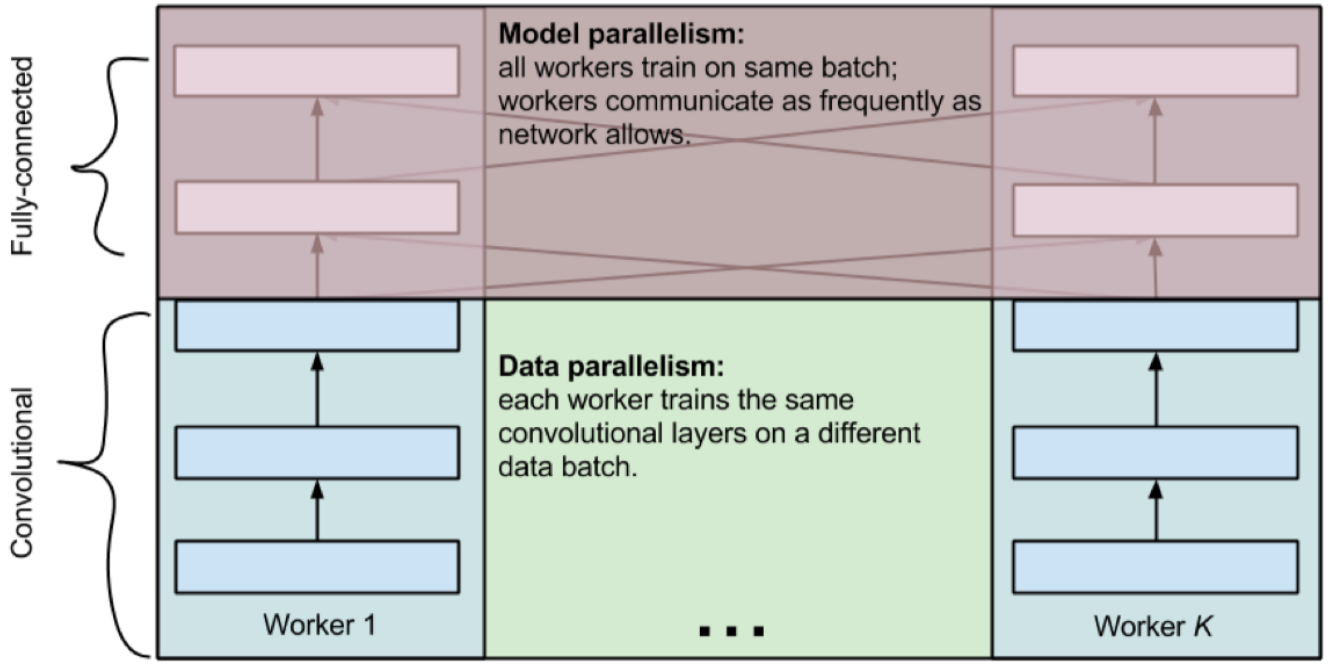
- Fully-connected layers
  - 5-10% of the computation
  - 95% of the parameters
  - Small intermediate activations

Tensor model parallelism

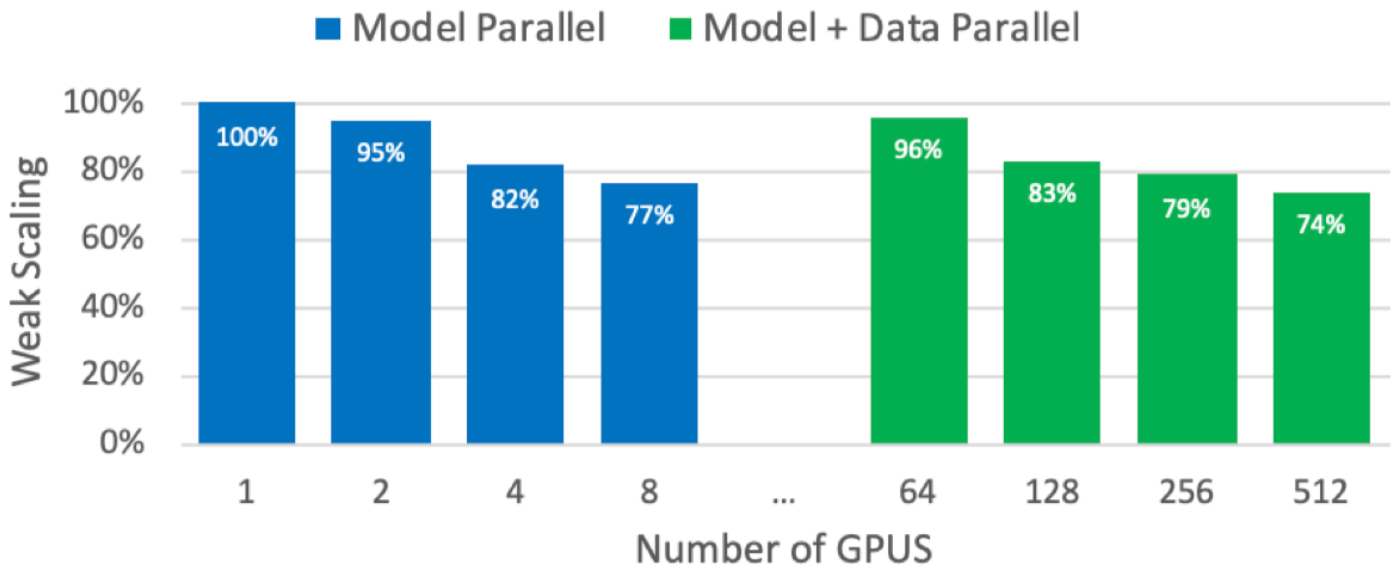
- **Discussion: how to parallelize CNNs?**

Parallelizing CNNs

- Data parallelism for convolutional layers
- Tensor model parallelism for fully-connected layers



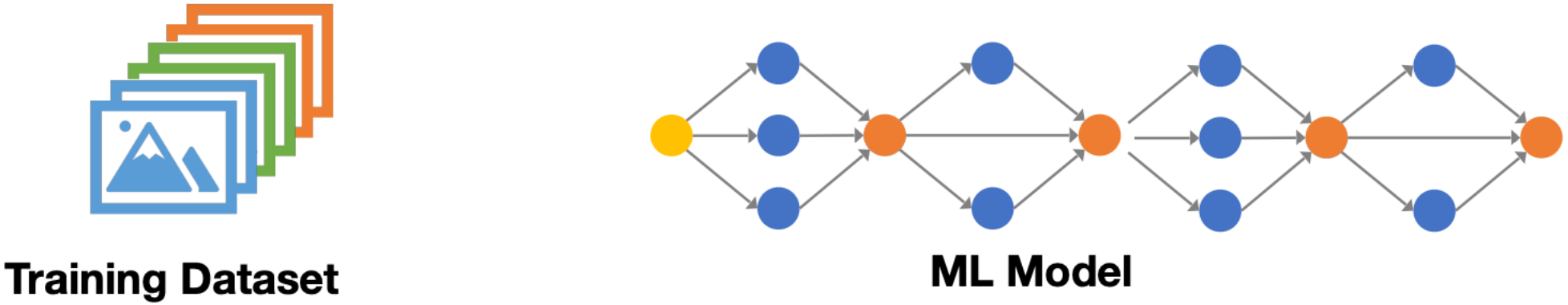
Parallelizing large models



Scale to 512 GPUs by combining data and model parallelism

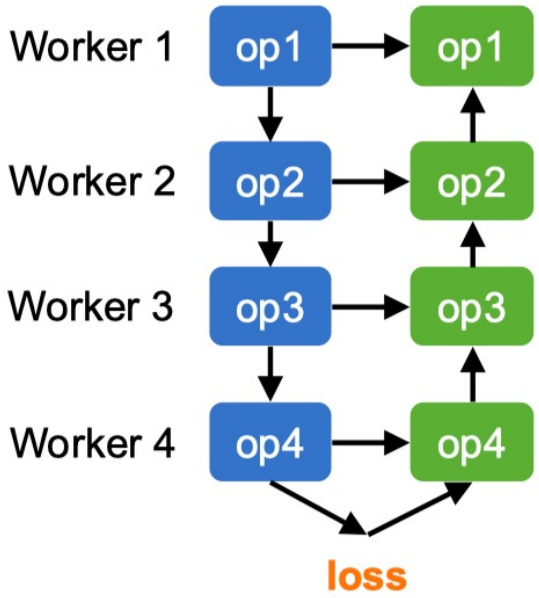
Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism.

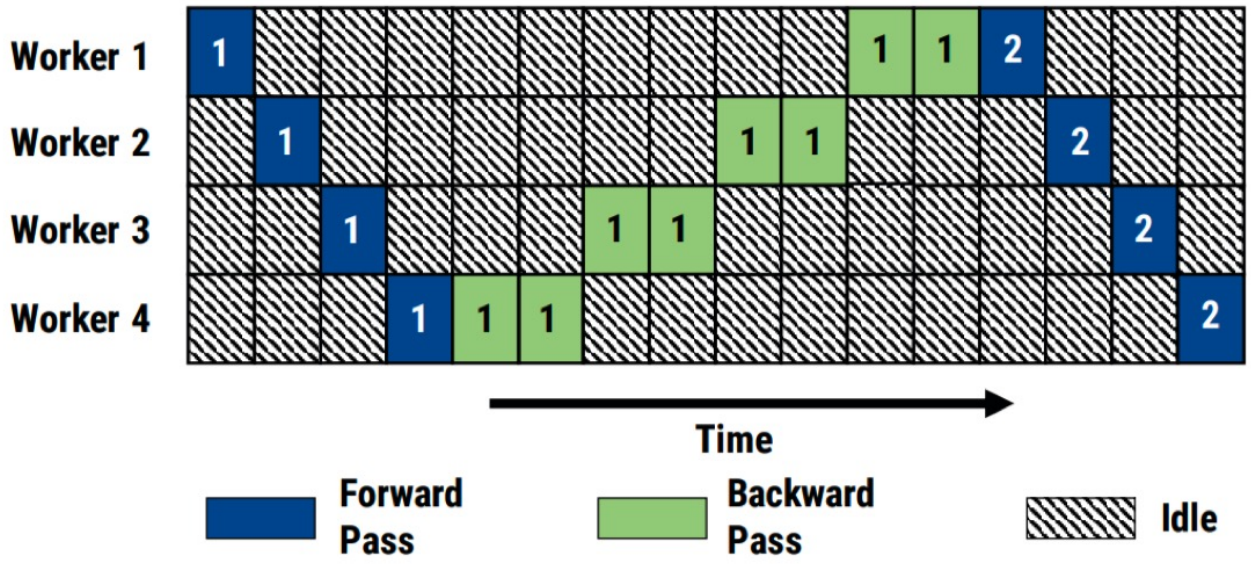
Pipeline model parallelism



Pipeline model parallelism

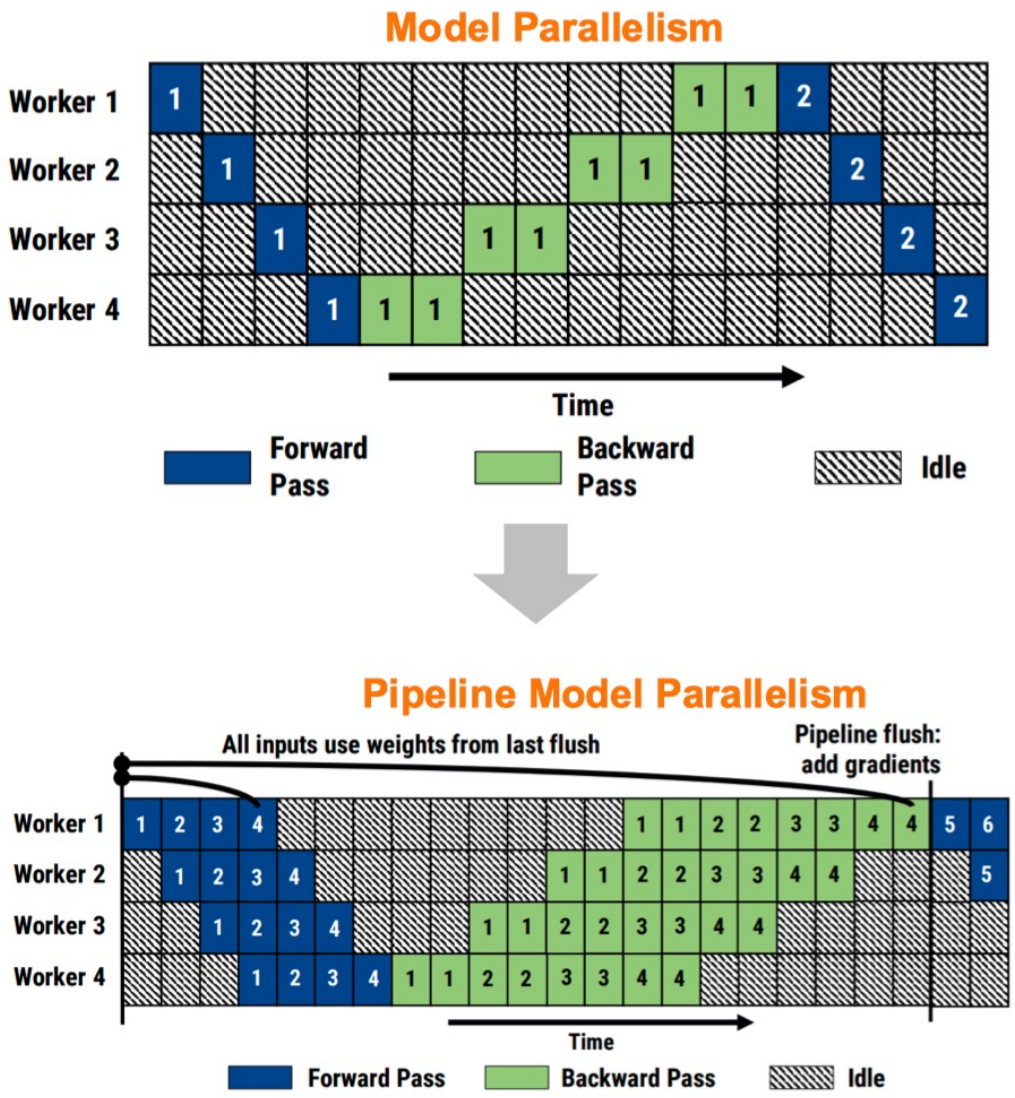
- Under-utilization of compute resources
- Low overall throughput due to resource utilization





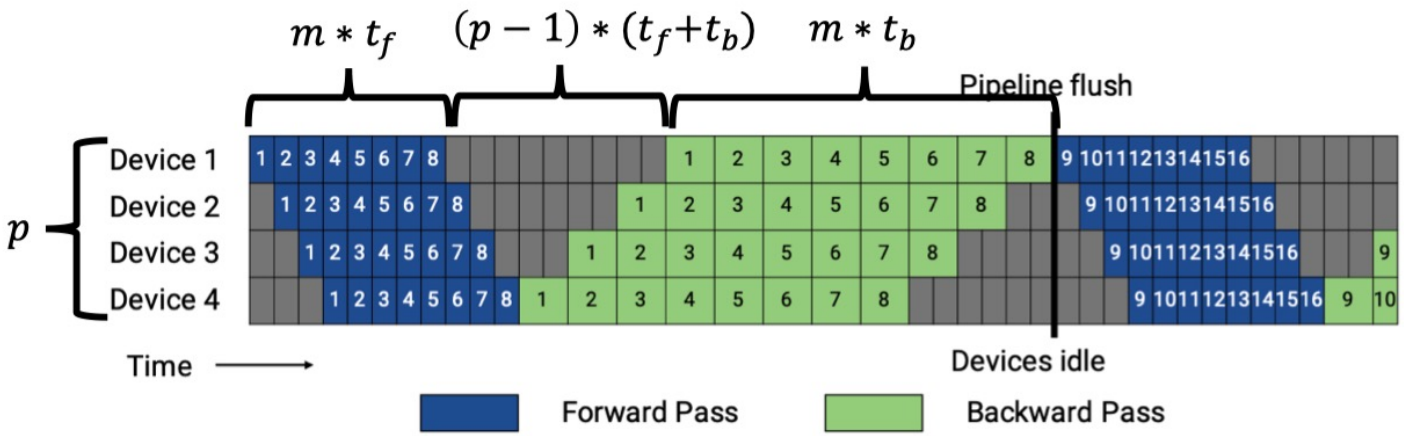
Pipeline model parallelism

- **Mini-batch**: the number of samples processed in each iteration
- Divide a mini-batch into multiple **micro-batches**
- Pipeline the forward and backward computations across micro-batches



Efficiency of pipeline model parallelism

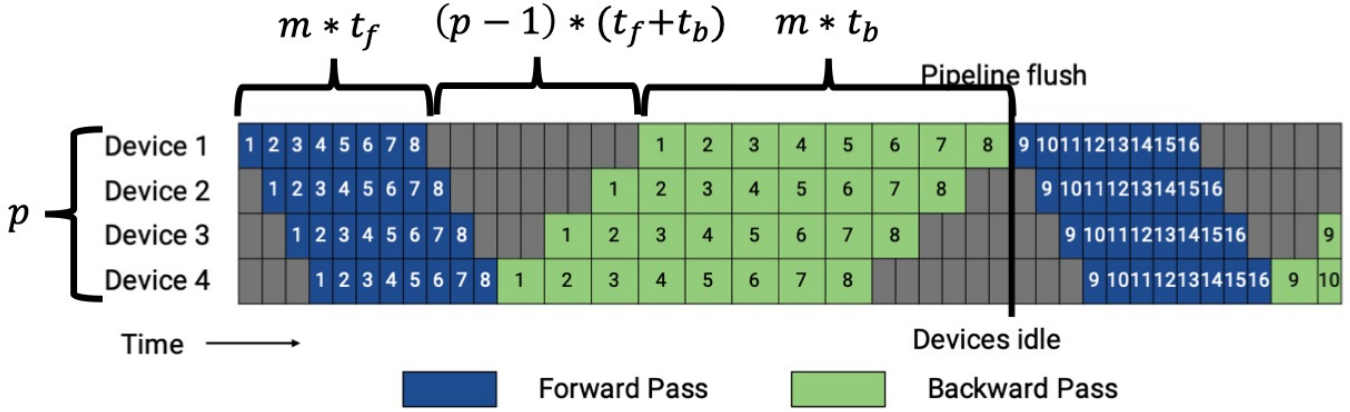
- m : micro-batches in a mini-batch
- p: number of pipeline stages
- All stages take t_f/ t_b to process a forward (backward) micro-batch



$$\textbf{BubbleFraction} = \frac{(p-1) * (t_f+t_b)}{m * t_f + m * t_b} = \frac{p-1}{m}$$

Efficiency of pipeline model parallelism

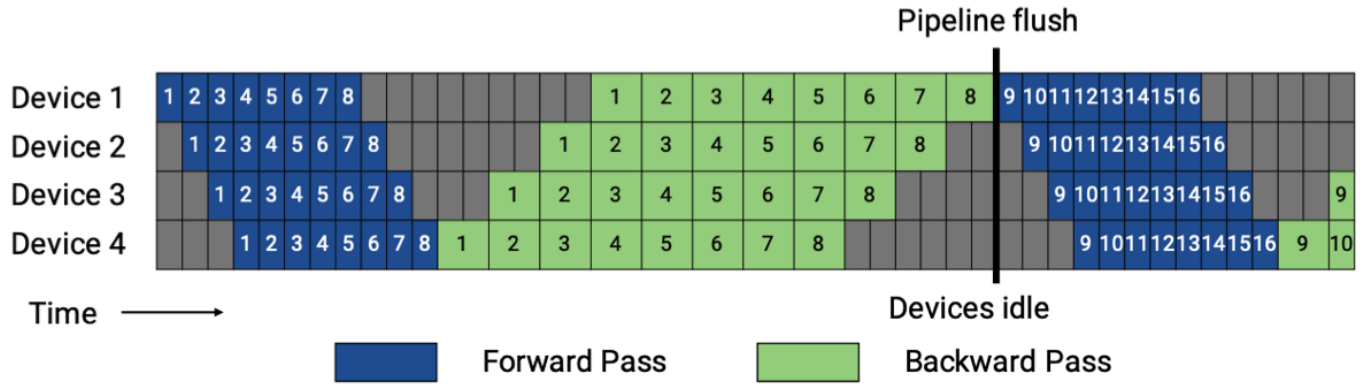
- m : number of micro-batches in a mini-batch
  - Increase mini-batch size or reduce micro-batch size
  - Caveat: large mini-batch sizes can lead to accuracy loss; small micro-batch sizes reduce GPU utilization
- p: number of pipeline stages
  - Decrease pipeline depth
  - Caveat: increase stage size



$$\textbf{BubbleFraction} = \frac{(p-1) * (t_f+t_b)}{m * t_f + m * t_b} = \frac{p-1}{m}$$

Pipeline model parallelism: Memory issues

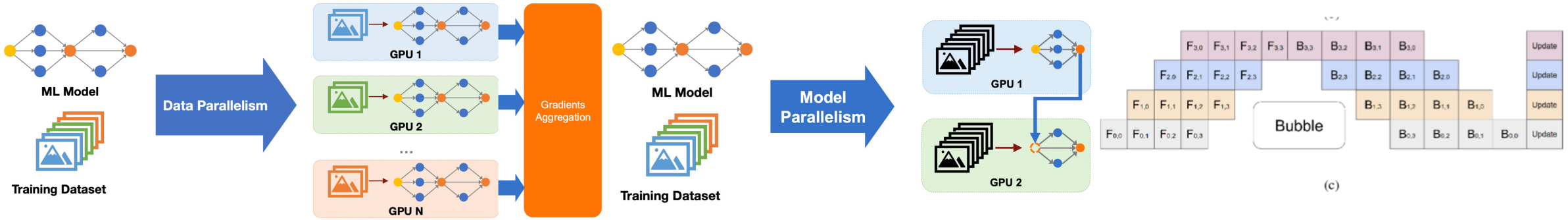
- An issue: we need to keep the intermediate activations of **all micro-batches** before back propagation



How can we reduce memory footprint?

Limit the number of in-flight micro-bathes (e.g., 1F1B pipeline schedule)

Parallelism summary

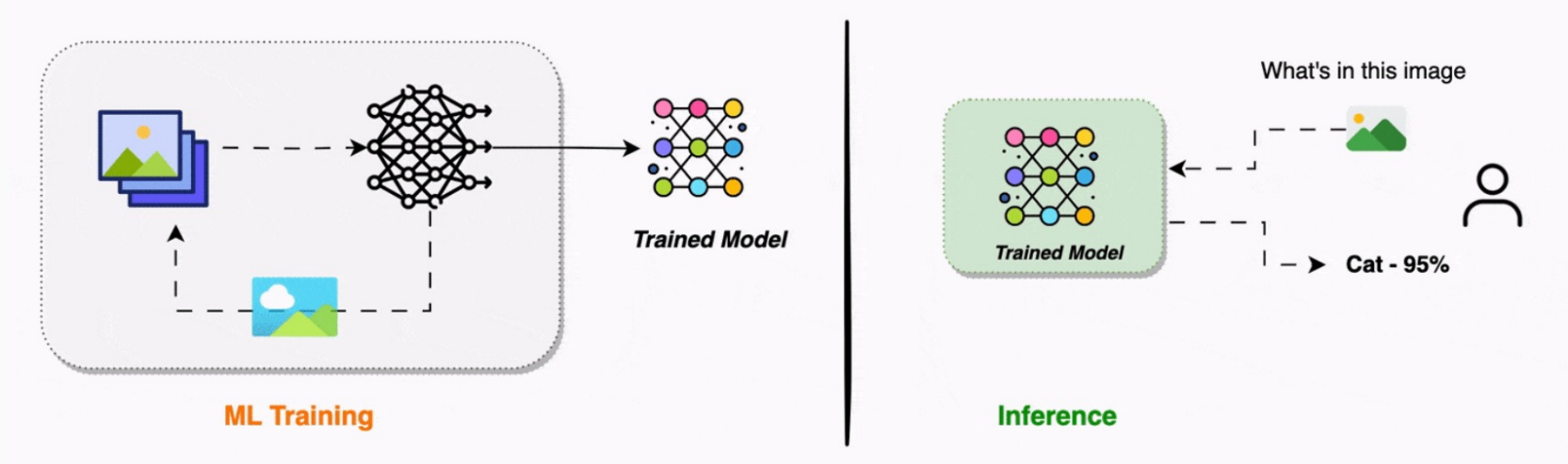


	Data Parallelism	Tensor Model Parallelism	Pipeline Model Parallelism
Pros	✓ Massively parallelizable ✓ Require no communication during forward/backward	✓ Support training large models ✓ Efficient for models with large numbers of parameters	✓ Support large-batch training ✓ Efficient for deep models
Cons	❖ Do not work for models that cannot fit on a GPU ❖ Do not scale for models with large numbers of parameters	❖ Limited parallelizability; cannot scale to large numbers of GPUs ❖ Need to transfer intermediate results in forward/backward	❖ Limited utilization: bubbles in forward/backward

ML serving

ML serving

- Deploy the trained model as an online model inference service

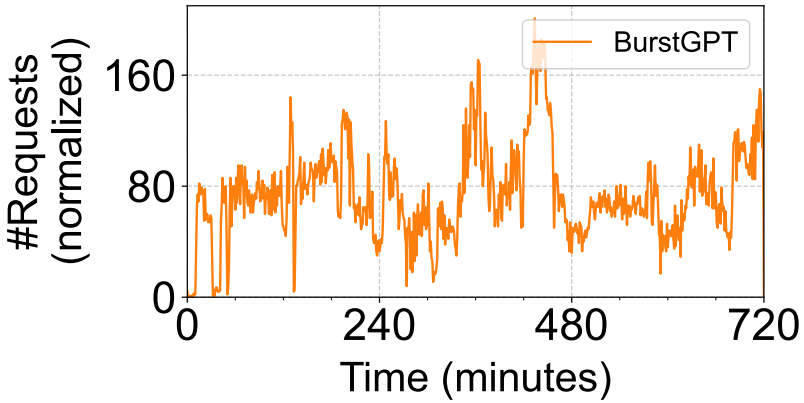


Just forward pass

- Handle a massive volume of inference requests
  - Tens of trillions per day in Facebook [1]; up to 90% of cost in AWS AI cloud [2]

[1] Hazelwood et al., “Applied machine learning at Facebook: a datacenter infrastructure perspective,” in HPCA 2018.
[2] Deliver high-performance ML inference with AWS Inferentia, AWS ReInvent 2019

Goals for model inference



Auto-scale to dynamic user requests

Performance/SLO-aware

- Meeting response-time SLOs (Service-Level Objectives)
  - e.g., 98% of the requests should be served in 500 ms
- Attaining high throughput, measured by requests per second (RPS)

Cost-effective

- Minimizing the service cost by improving resource utilization

Batching

Similar to training, we can process inference requests in batches

Increasing the batch size helps improve parallelism

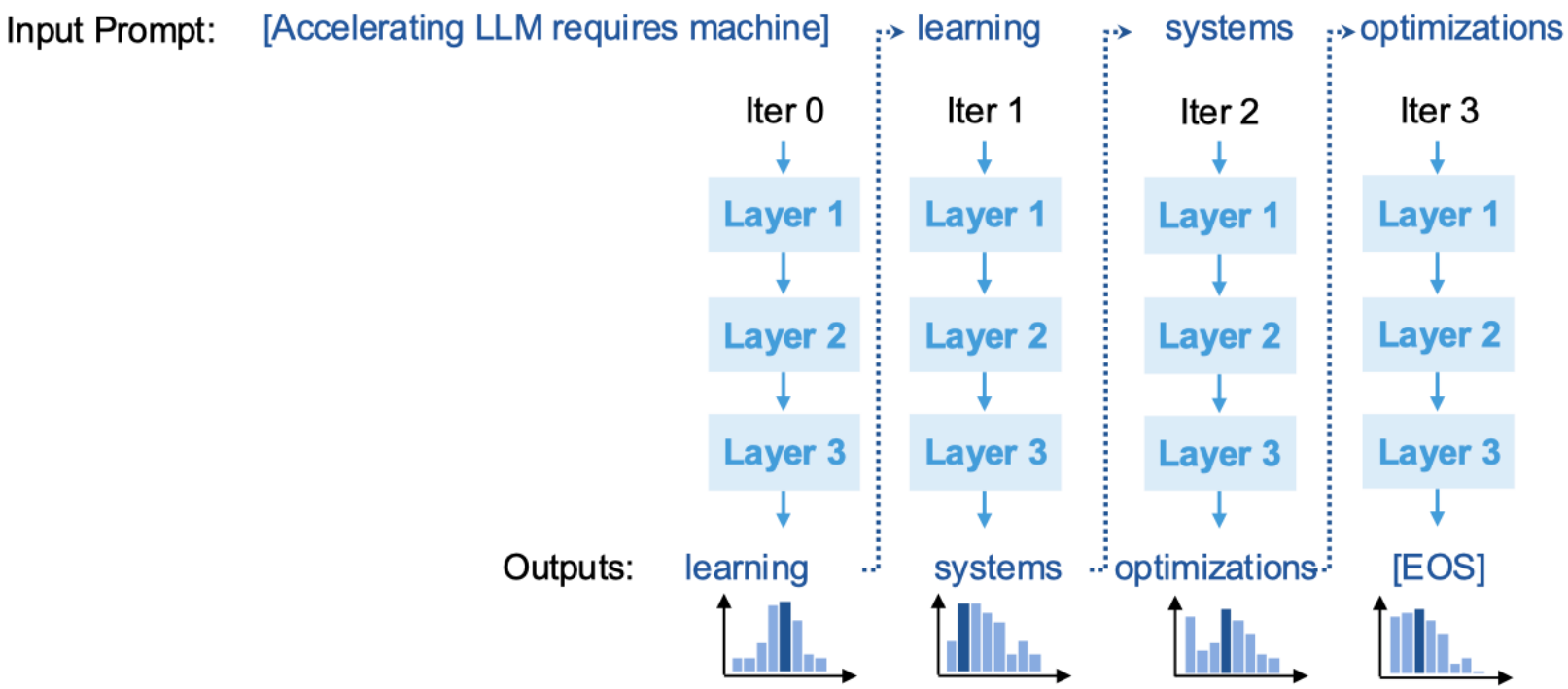
- Provide more work to parallelize and improve throughput

But increasing the batch size can make us do more work before we can return an answer for any individual example

- Can negatively affect latency

Generative LLM inference

Autoregressive pattern in Large Language Model (LLM) inference

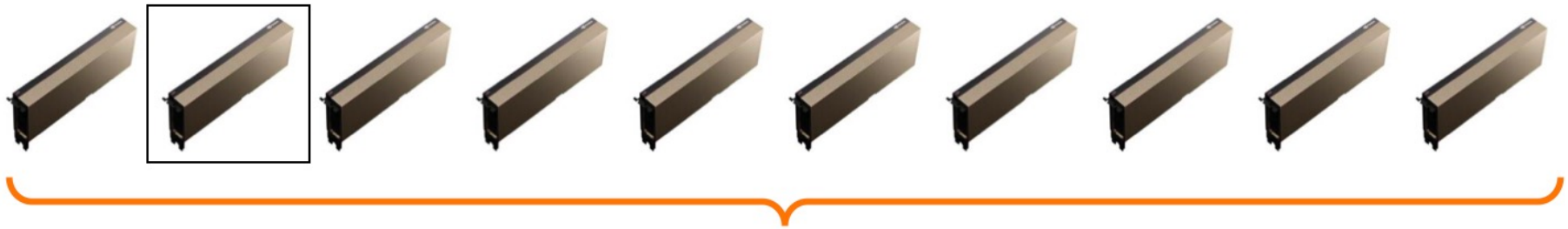


Generative LLM inference

- **Pre-filling phase** (0-th iteration):
  - Process ***all*** input tokens at once
- **Decoding phase** (all other iterations):
  - Process a ***single*** token generated from previous iteration
  - Use attention keys & values of all previous tokens

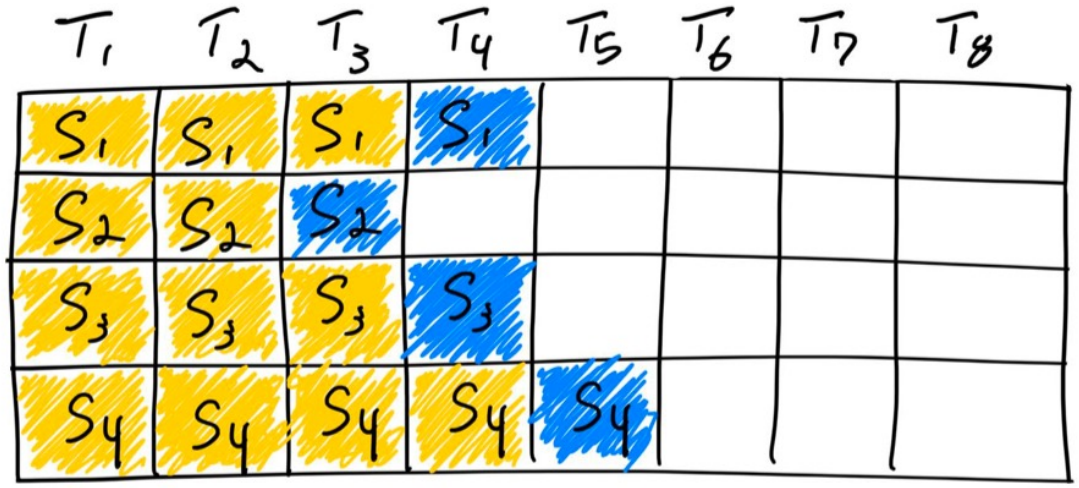
- Key-value cache:
  - Save attention keys and values for the following iterations to avoid recomputation

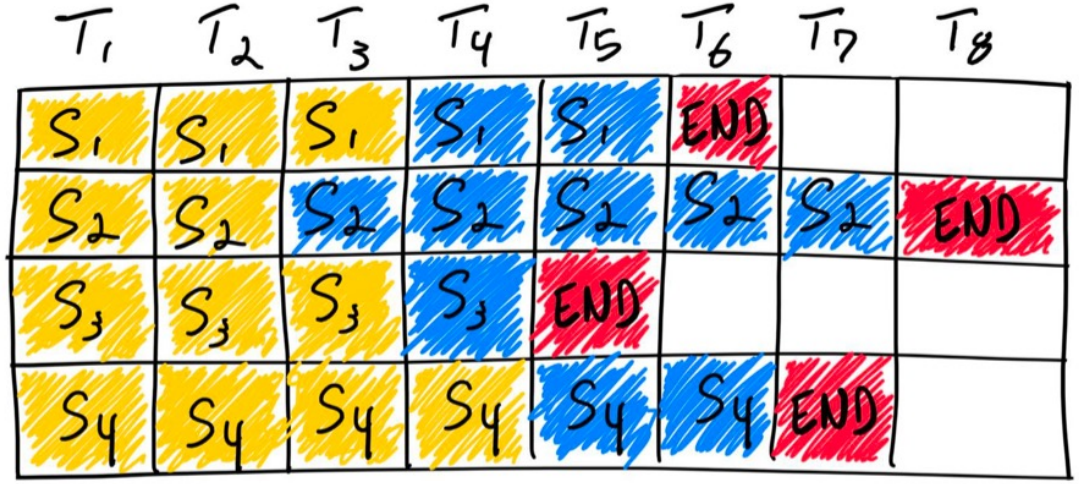
Generative LLM inference



- **At least ten** A100-40GB GPUs to serve 175B GPT-3 in half precision
- Generating 256 tokens takes **~20 seconds**
- Cannot process many requests in parallel
  - Per-request key/value cache takes **3GB GPU memory**

Batching in LLM inference

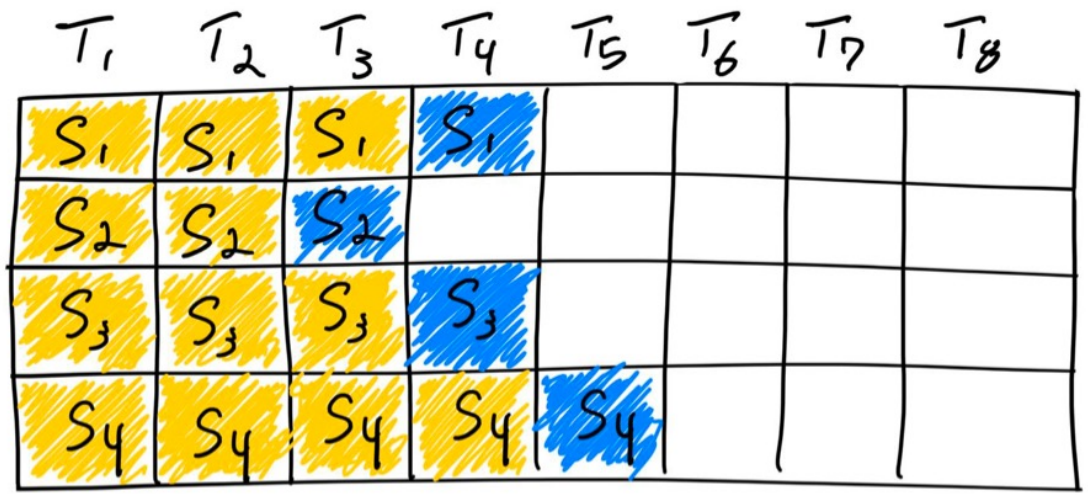


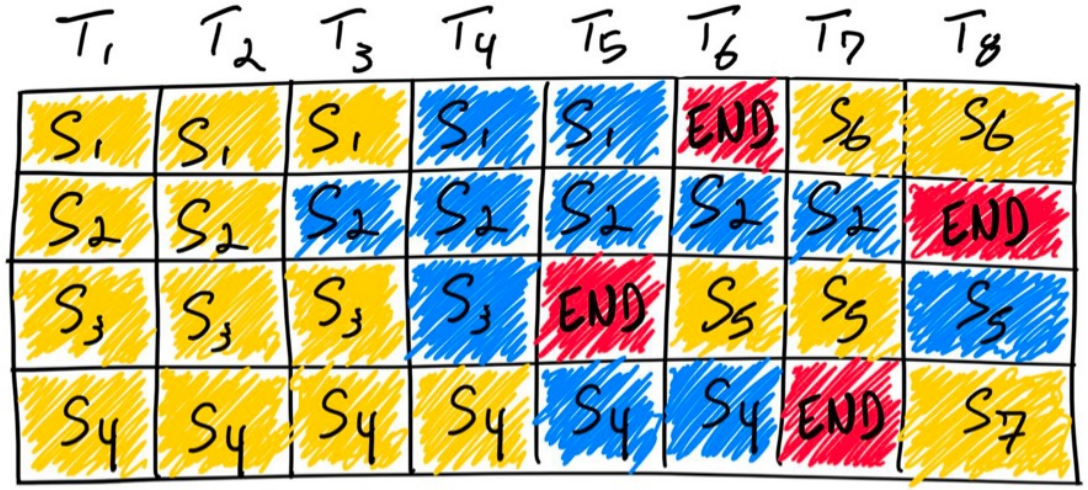


Issues with static batching:

- Requests may complete at different iterations
- Idle GPU cycles
- New requests cannot start immediately

Continuous batching





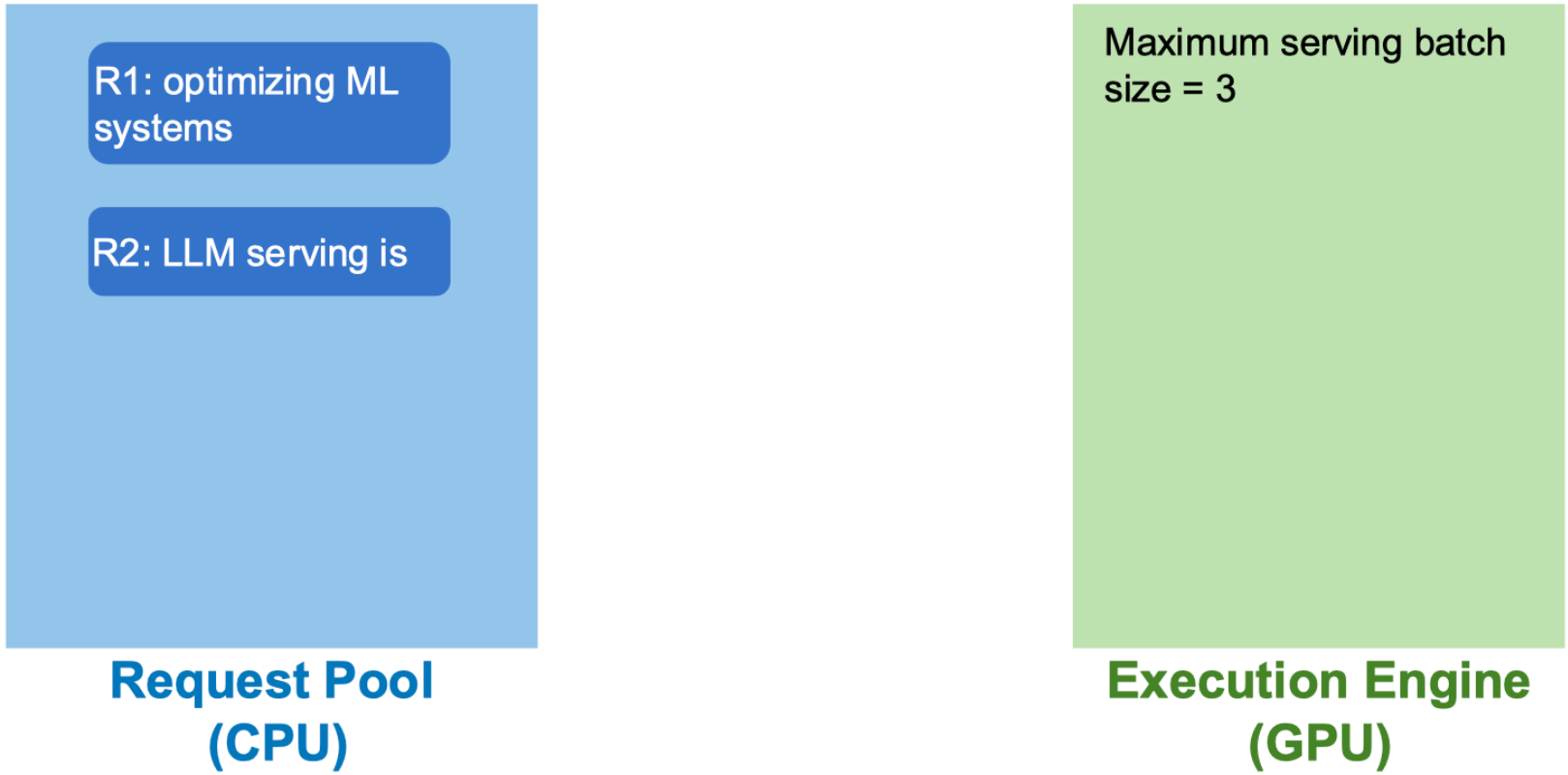
Benefits:

- Higher GPU utilization
- New requests can start immediately

Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI'22

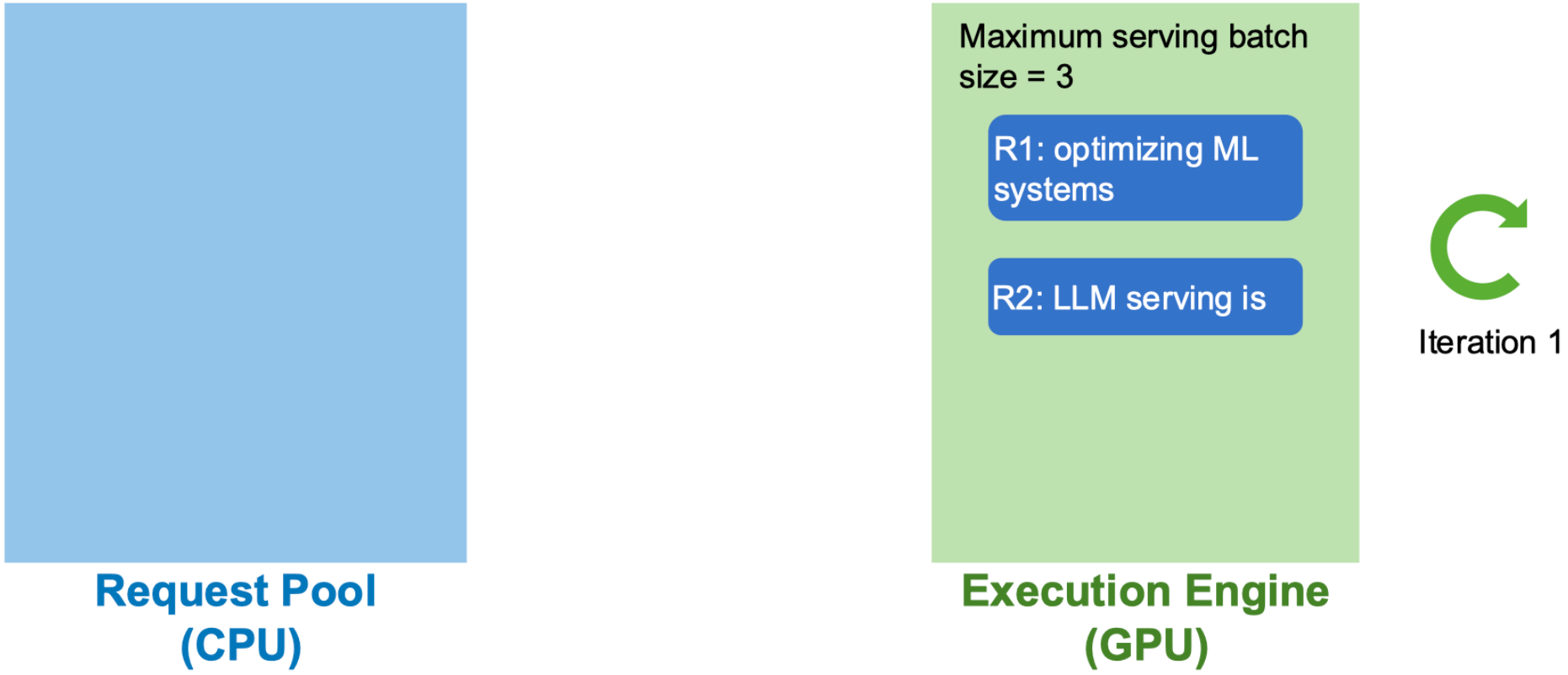
Continuous batching

- Receives two new requests R1 and R2



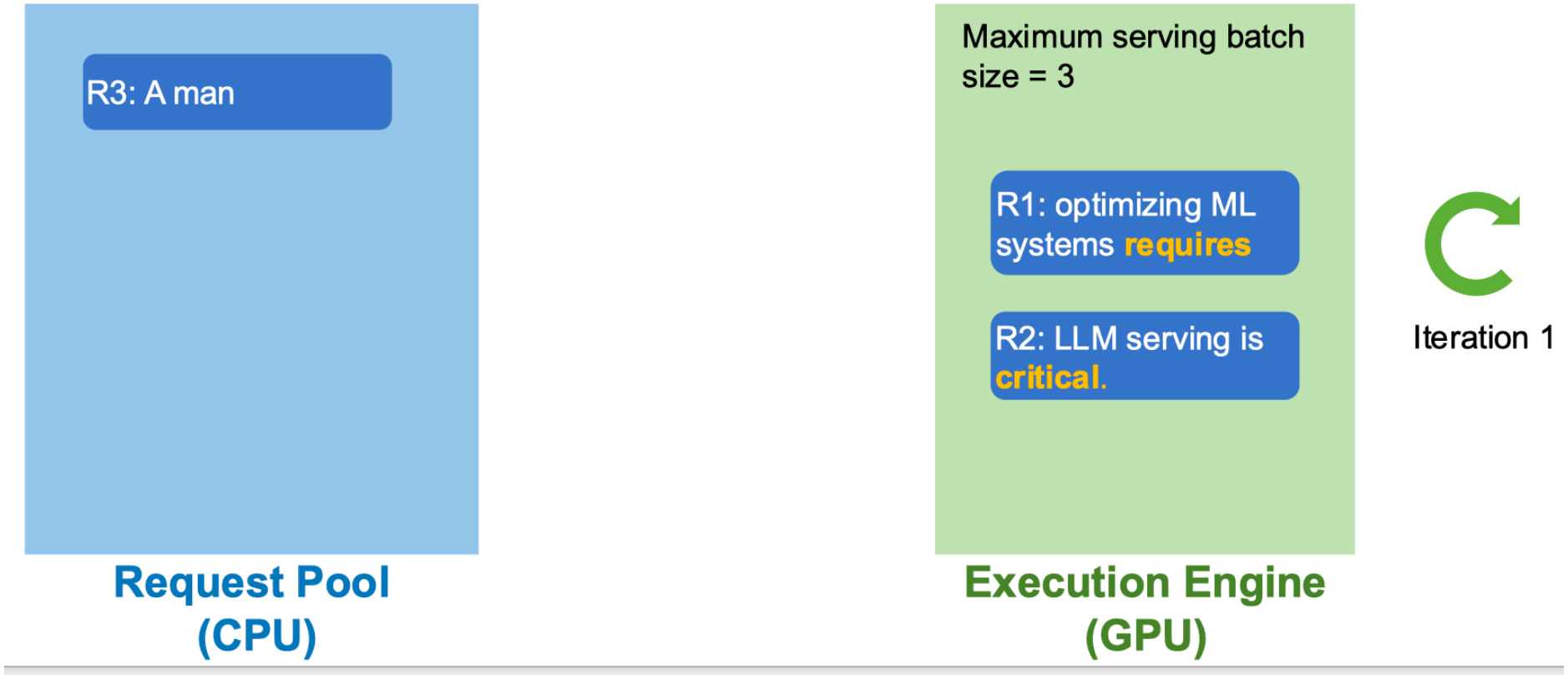
Continuous batching

- Iteration 1: decode R1 and R2



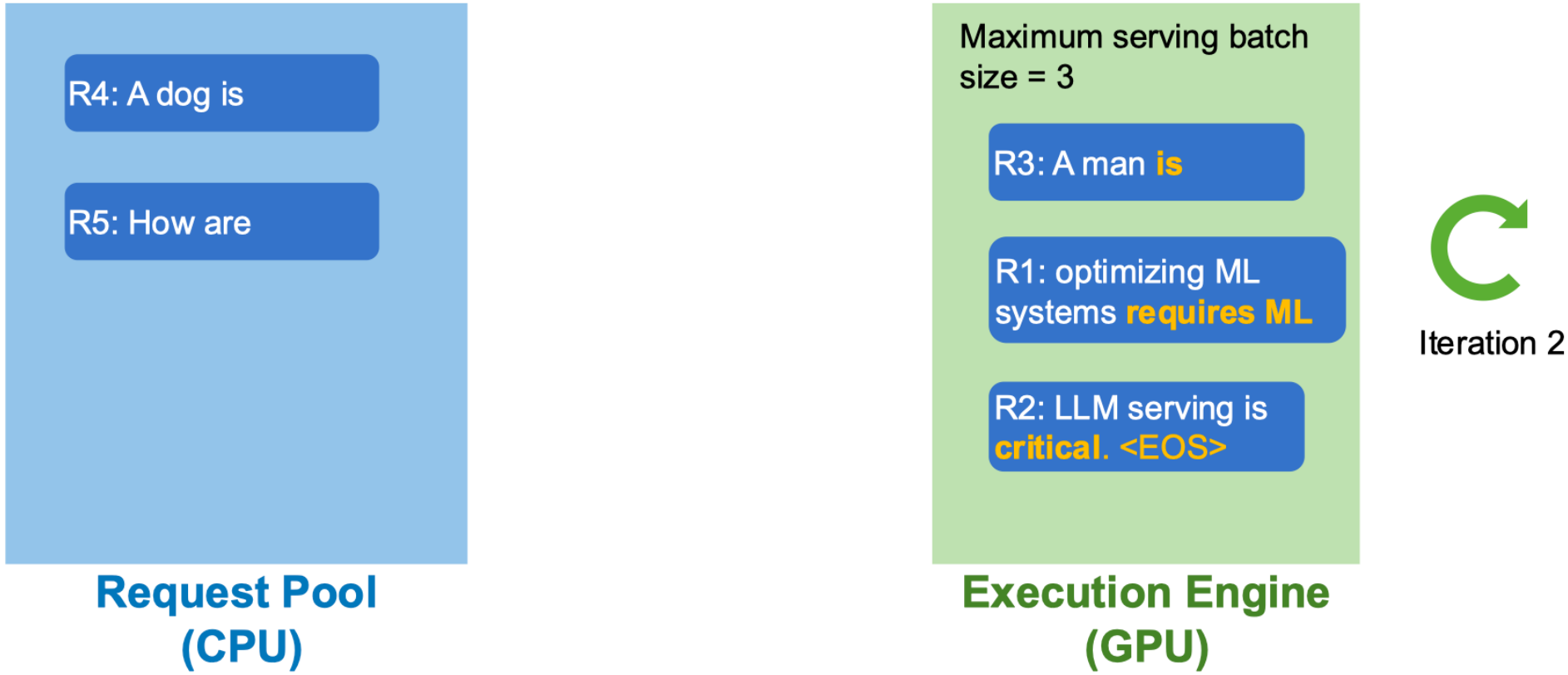
Continuous batching

- Receive a new request R3; finish decoding R1 and R2



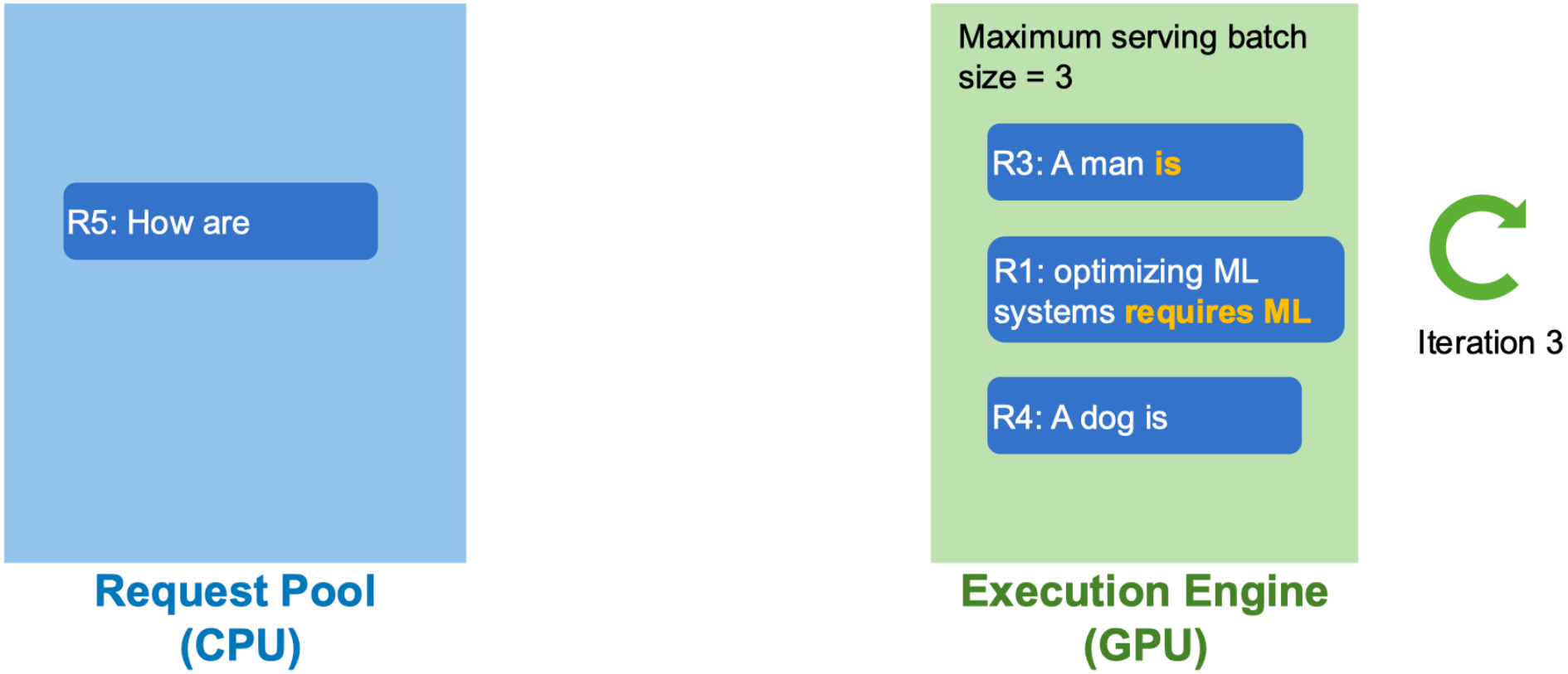
Continuous batching

- Iteration 2: decode R1, R2, R3; receive R4, R5; R2 completes



Continuous batching

- Iteration 3: decode R1, R3, R4



Continuous batching

Advantages

- Handle early-finished and late-arrived requests more efficiently
- Higher GPU utilization

Limitations?

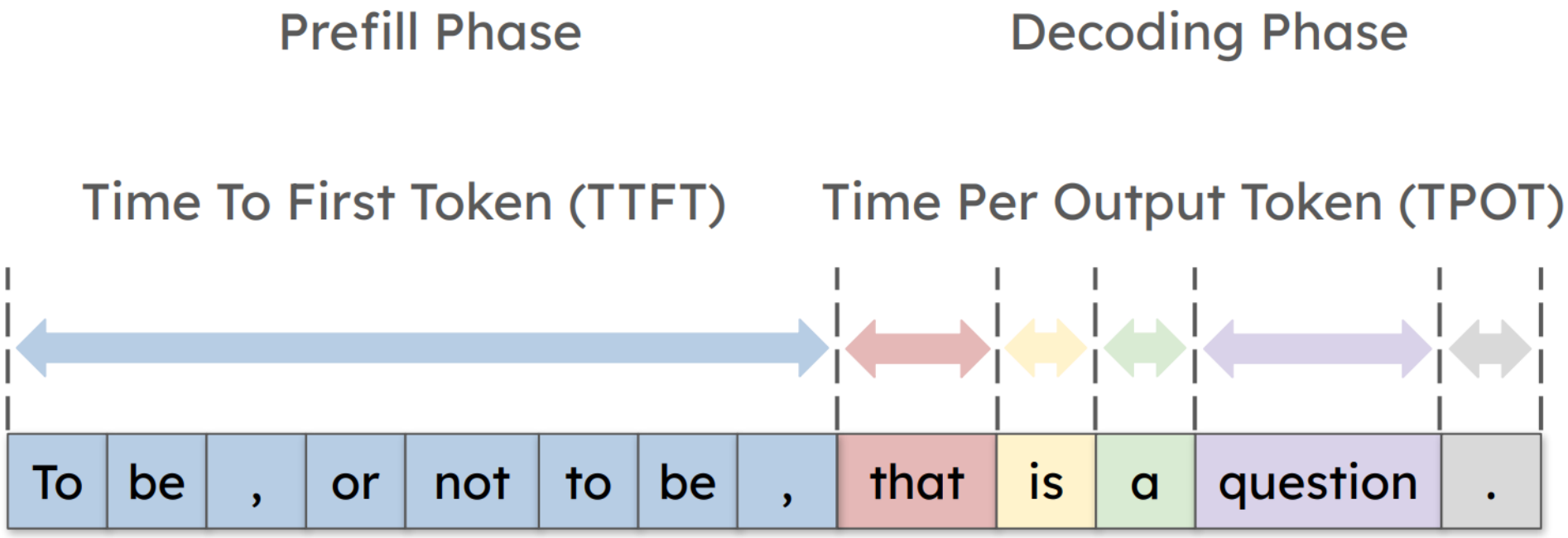
Prefill and decoding may interference with each other

State-of-the-art LLM systems

LLM systems

- Disaggregated LLM inference
- Fast LLM scaling

LLM inference latency

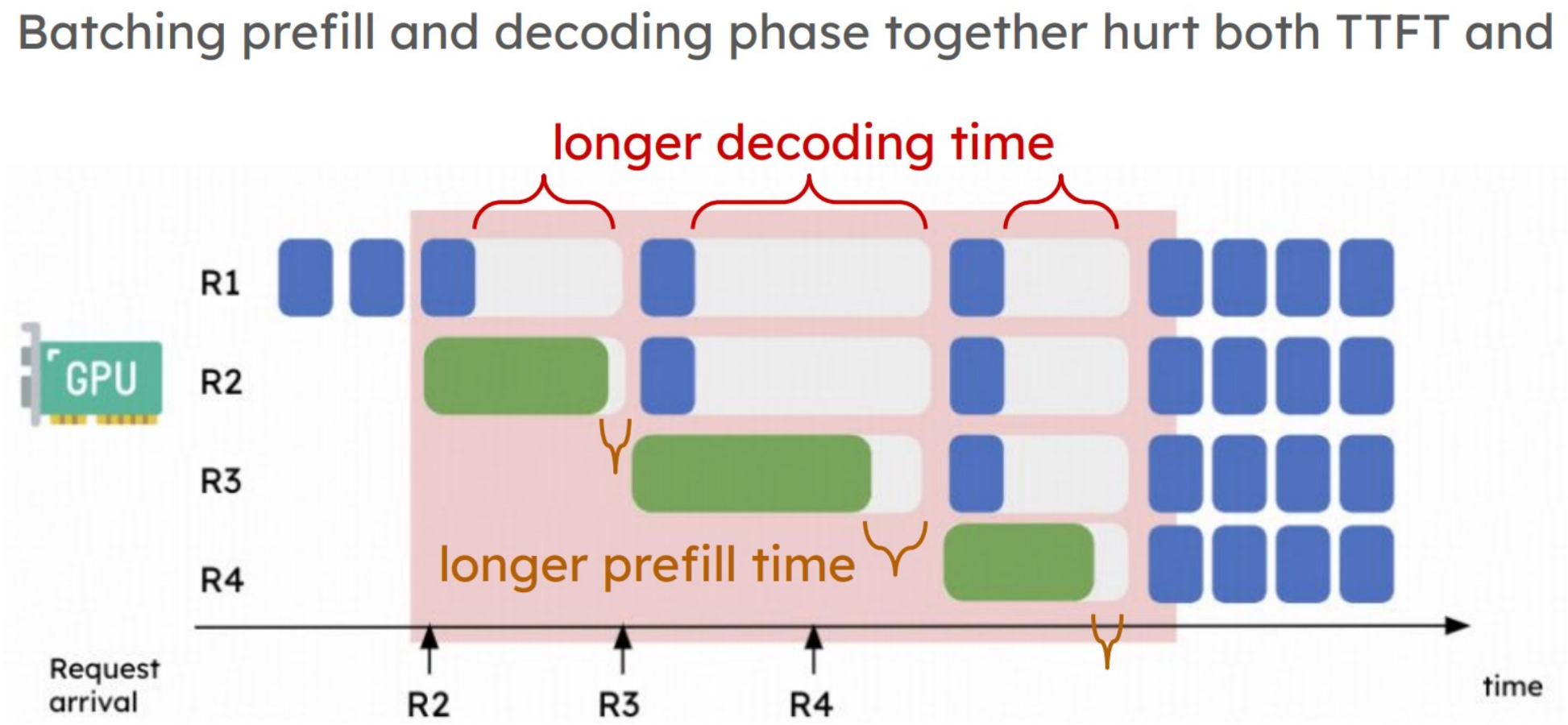


Latency = TTFT + TPOT * #Token

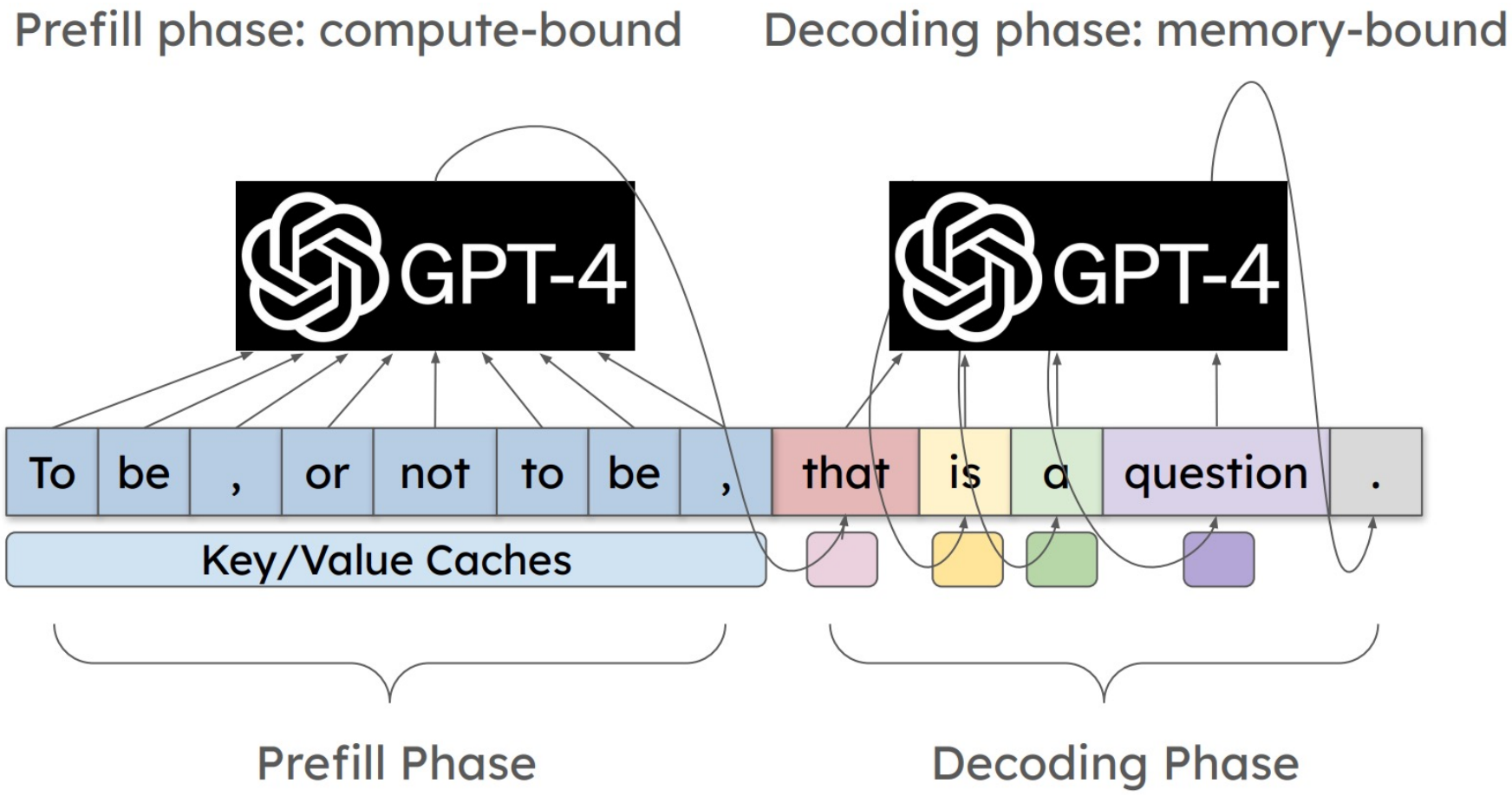
LLM inference latency

	TTFT	TPOT
Chat	Low (< 1s)	Match read speed (~ 100ms)
Search	Very Low (~ 200ms)	Match read speed (~ 100ms)
Program	Very Low (~ 200ms)	Very Low (~ 50ms)

Prefill-decoding interference

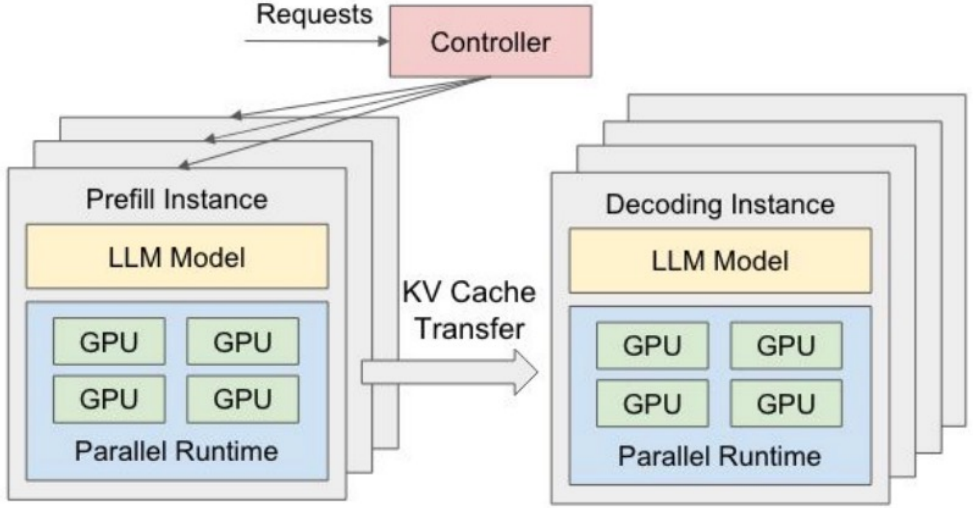


Prefill-decoding interference

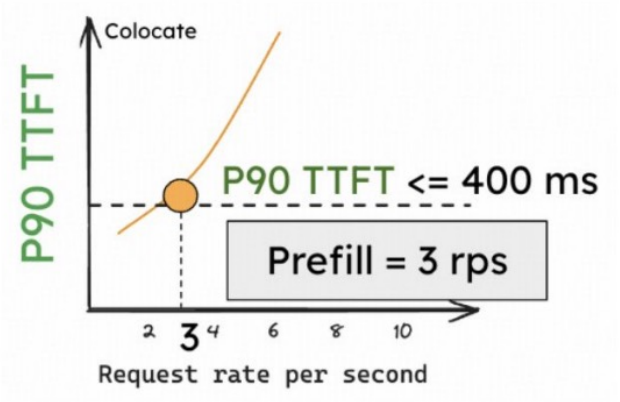


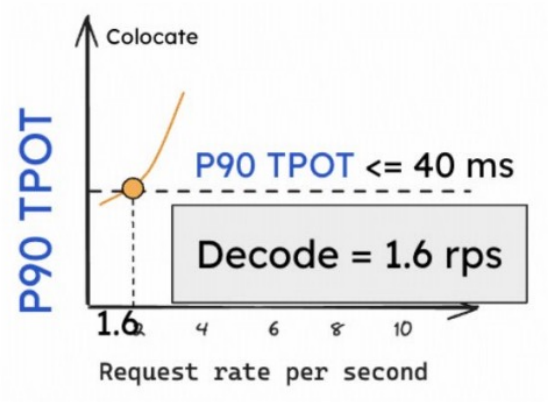
Disaggregated prefill and decoding

- Prefill-Decoding interference is immediately eliminated
- Naturally divide the SLO satisfaction problem into two optimizations:
  - Prefill instance optimizes for TTFT.
  - Decoding instance optimizes for TPOT.
  - Choose the most suitable parallelism and resource allocation for each phase.



Disaggregated prefill and decoding





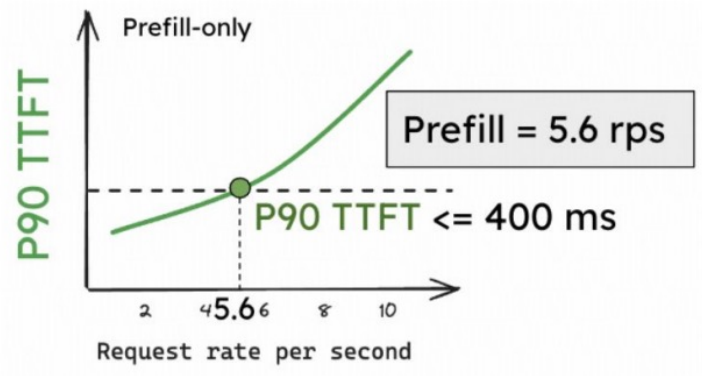
Colocation

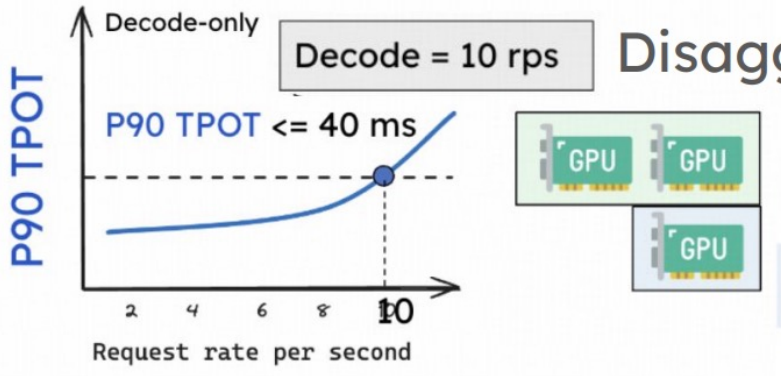
GPU

Max System rps

= Min(Prefill, Decode)

= 1.6 rps / GPU





Disaggregation (2P1D)

GPU GPU GPU

Max System rps

= Min (5.6 x **2**, 10) rps / 3 GPU

= 3.3 rps / GPU

Challenges of disaggregation

- Communication overhead for KV-Cache transmission
- Hard to optimize end-to-end performance due to:
  - the workload pattern
  - SLO requirements
  - parallelism strategies and resource allocation

Refer to recent P/D disaggregation papers

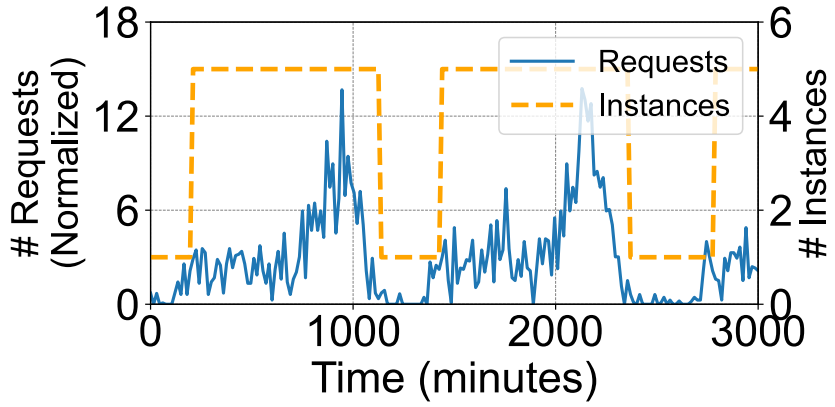
- “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving” in OSDI’24

Fast LLM scaling

Need of fast scaling: meet SLOs under highly dynamic requests

- Large resource footprint
  - Large models require hundreds of GBs to multiple TBs of memory
- Model proliferation
  - >2M models on Hugging Face
- Dynamic and bursty request patterns
  - Demand can spike 10x within minutes

Hard to achieve scalable inference!

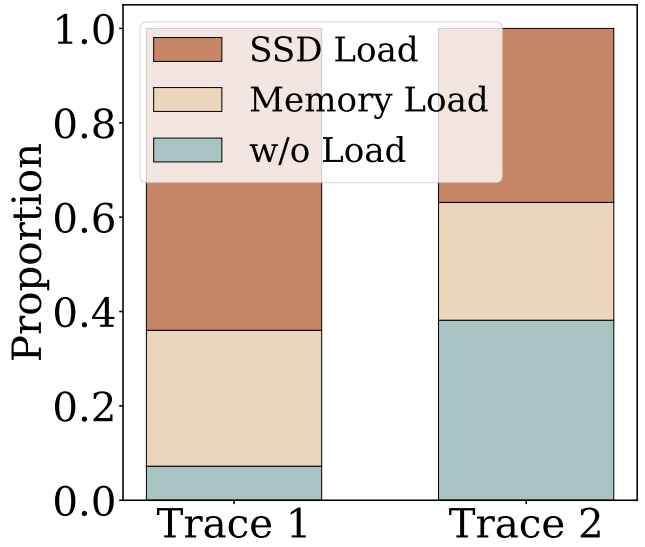


Existing solutions to serverless inference

- Load remote models (e.g., from S3)
  - Slow, bandwidth contention under concurrent model fetching

- Overprovision GPUs
  - Expensive, resource waste during idle periods

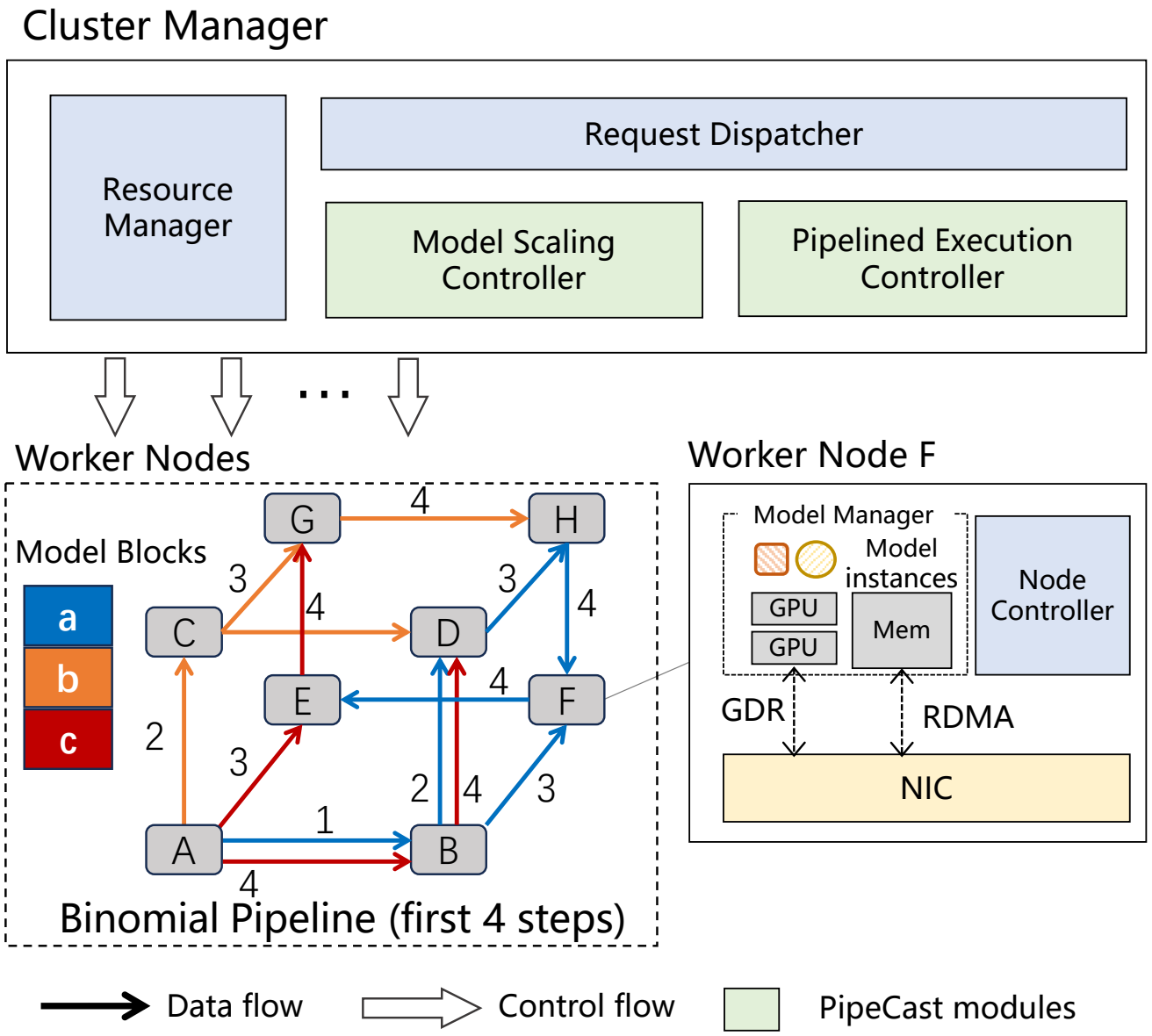
- Cache models in host memory and SSDs
  - Keep-alive times in memory for most models < 15 seconds
  - High host memory cache miss rates lead to fallback on slow SSDs



Key insights

- Leverage high-speed interconnects in GPU clusters (200–400 Gbps with RDMA) to distribute models between nodes
  - Fast model multicast without the need of caching many models in memory

- Inference execution can begin while models are being loaded
  - Execute-while-load, collaborative inference during model multicast



FaaScale overview

- Binomial-pipeline-based[1] model multicast
- Cooperative inference across nodes during model multicast
- GDR-based model transfer from GPU/host memory

[1]Behrens. J et al., “RDMC: A Reliable RDMA Multicast for Large Objects”, DSN’18.

Key challenges and design overview

Coordinated model multicast and inference execution

- Inference-aware model multicast
- Pipelined, collaborative inference execution

Cross-storage-tier model management

- Locality-driven model startup
- Efficient memory management

Refer to the paper for details

- “FaaScale: Unlocking Fast LLM Scaling for Serverless Inference” in MLSys’26

Reference & Credits

- Some slides adapted from course slides of 15-779 in CMU
- Some slides adapted from course slides of DSC204A in UCSD
- Yinmin Zhong et al, “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving” in OSDI’24